

Régression logistique et classification

un-contre-tous

Modélisation, estimation, calcul numérique et évaluation

Sylvain Maitre

31 juillet 2026

Résumé

Un fichier contient mille six cents élèves. De chacun, on connaît treize notes et la maison. Il faut en tirer une règle qui donne sa maison à un élève dont on ne connaît que les notes.

La règle construite ici n'annonce pas directement une maison : elle calcule une probabilité. Cette probabilité vient d'une somme pondérée des notes, ramenée entre zéro et un par la fonction logistique. Les poids de cette somme sont les inconnues. Le principe du maximum de vraisemblance les fixe : on retient les poids sous lesquels les maisons effectivement observées deviennent les plus probables. Cette exigence s'écrit comme un coût à réduire, dont la pente indique en chaque point la correction à appliquer aux poids ; répéter la correction fait descendre le coût jusqu'à son minimum.

Quatre maisons demandent quatre exécutions de ce procédé, chacune posant la question « celle-ci, ou bien l'une des trois autres ». Un élève reçoit alors quatre scores et se voit attribuer la maison du plus grand ; softmax n'est pas nécessaire pour décider. Le reste du travail porte sur les mesures : quelles notes retenir, quoi mettre dans les cases vides, à quoi reconnaître un résultat qui vaudra encore pour des élèves jamais vus. Cette part occupe autant de place que le modèle lui-même, un classifieur qui reconnaît sans faute les élèves de son propre fichier n'ayant encore rien démontré.

Ce texte a été rédigé avec GPT et Claude selon mes instructions et éléments fournis. Le résultat n'est pas formidable mais il est lisible et peut apporter des éléments de compréhension. Il est mis à disposition sous licence CC-BY-SA 4.0.

Table des matières

1	Problème statistique et chaîne de traitement	1
1.1	Décision et calibration probabiliste	3
2	Données, préparation et protocole expérimental	4
2.1	Matrice de design	4
2.2	Valeurs manquantes et standardisation	5
2.3	Partition et déséquilibre des classes	8
2.4	Préservation des effectifs par classe	8
3	Modèle logistique binaire	10
3.1	Interprétation par les log-cotes	10
3.2	Frontière de décision	12
4	Estimation par maximum de vraisemblance	14
4.1	Gradient et correction des coefficients	17
4.2	Convexité, unicité et séparabilité	17
4.3	Régularisation facultative	19
5	Optimisation et calcul numérique	22
5.1	Influence du taux d'apprentissage dans le cas scalaire	24
5.2	Formules numériquement stables	25
6	Extension multi-classes un-contre-tous	27
7	Évaluation et expérience reproductible	31

7.1	Matrice de confusion et mesures	31
7.2	Protocole minimal d'évaluation	33
7.3	Validation croisée : plis et répétitions	34
8	Entraînement et prédiction	37
8.1	Colonnes du fichier	37
8.2	Best Hand et Birthday	38
8.3	Redondance	38
8.4	Pouvoir discriminant	39
8.5	Valeurs manquantes	40
8.6	Matrice de design	40
8.7	Le programme d'entraînement	41
8.8	Le fichier de poids	42
8.9	Primitives à écrire	42
8.10	Pseudo-code de logreg_train	43
8.10.1	Constantes	43
8.10.2	Imputation et statistiques de colonne	44
8.10.3	Construction de $\mathbf{X}$	44
8.10.4	Descente de gradient, maison par maison	45
8.11	Le programme de prédiction	47
8.12	Contrôles	49
9	Notations et lecture des formules	51
9.1	Indices, effectifs et ensembles	51
9.2	Une observation et ses caractéristiques	54
9.3	Classes observées, cibles binaires et prédictions	54
9.4	Paramètres, score et probabilité binaire	54
9.5	Vraisemblance, perte et optimisation	56
9.6	Probabilités et opérateurs	57
9.7	Lois et fonctions	57

9.8	Objets de l'optimisation	59
9.9	Statistiques descriptives	60
9.10	Tests et mesures d'évaluation	61
9.11	Correspondance avec les notations du sujet	62
10	Guide de lecture et ressources ouvertes	64
10.1	Premiers repères statistiques	64
10.2	Probabilités et calcul	64
10.3	Approfondir le formalisme	65
	Conclusion	66
A	Dérivations mathématiques	67
A.1	Inversion du logit	67
A.2	Gradient de la log-perte	67
A.3	Convexité de la log-perte	68
A.4	Direction de descente	68

Chapitre 1

Problème statistique et chaîne de traitement

Le fichier fournit des élèves dont la maison est connue. Le modèle cherche des régularités reliant leurs notes à leur maison, puis les applique à un élève absent du fichier. Toute l'analyse porte sur l'écart entre la réussite sur les élèves déjà vus et la réussite sur de nouveaux élèves.

On observe m élèves. L'élève d'indice i est décrit par n variables numériques $\mathbf{x}_i \in \mathbb{R}^n$ et par une maison c_i appartenant à

$$\mathcal{C} = \{\text{Gryffindor, Hufflepuff, Ravenclaw, Slytherin}\}, \quad K = |\mathcal{C}| = 4.$$

Le but est une règle $f : \mathbb{R}^n \rightarrow \mathcal{C}$ dont l'erreur reste faible sur des élèves nouveaux, issus du même mécanisme de génération des données. La performance sur l'échantillon observé sert d'indicateur intermédiaire.

Le terme *régression* désigne ici la modélisation d'un log-rapport de chances par une fonction affine. La variable réponse reste qualitative, ce qui range le problème dans la classification [1, chap. 4].

La table 1.1 montre quatre lignes du fichier. Ces valeurs passent par l'imputation et la standardisation du chapitre 2 avant d'entrer dans le modèle. La colonne House porte la réponse à prédire.

élève	Astronomy	Herbology	Charms	Flying	House
1	−487,9	5,7	−232,8	−26,9	Ravenclaw
2	−552,1	−6,0	−252,2	−113,5	Slytherin
3	−366,1	7,7	−227,3	30,4	Ravenclaw
4	697,7	−6,5	−256,8	200,6	Gryffindor

TABLE 1.1 – Quatre lignes du jeu d'entraînement, réduites à quatre colonnes de notes sur les treize disponibles. Une ligne représente une observation.

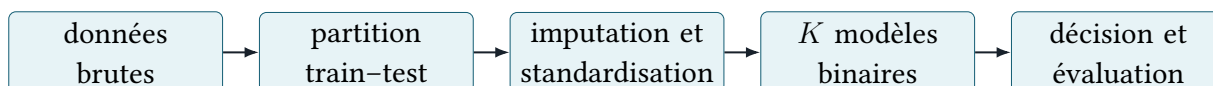


FIGURE 1.1 – Chaîne de traitement. Les statistiques de préparation sont estimées sur l'ensemble d'apprentissage, puis appliquées telles quelles aux données de test.

Ordre global : apprentissage, sélection et évaluation

1. **Partitionner.** Construire les indices d'apprentissage et de test, puis mettre le test à l'écart.
2. **Préparer.** Estimer imputation, moyennes et écarts-types sur l'apprentissage ; transformer les données avec ces valeurs.
3. **Organiser la sélection.** Réserver une validation ou construire des plis à l'intérieur des données disponibles pour l'apprentissage.
4. **Former les cibles OvR.** Pour chaque classe k , construire le vecteur binaire $\mathbf{y}_k$.
5. **Ajuster une configuration.** Pour chaque classe, initialiser θ_k , puis répéter scores, probabilités, gradient et mise à jour jusqu'au critère d'arrêt.
6. **Comparer les configurations.** Choisir α , λ , le seuil et les autres hyperparamètres sur la seule validation.
7. **Figurer le modèle.** Conserver la préparation, l'ordre des caractéristiques, les K vecteurs θ_k et l'ordre des classes.
8. **Prédire le test.** Appliquer la préparation conservée, calculer les K scores de chaque ligne et retenir leur maximum.
9. **Évaluer une fois.** Comparer prédictions et classes réelles du test ; produire matrice de confusion et mesures par classe.
10. **Archiver l'expérience.** Conserver graine, indices, hyperparamètres, historiques de convergence et résultats finaux.

Chaque cadre « sous-procédure » du document développe une opération appelée par cet ordre global : un calcul interne, un contrôle ou une méthode de sélection.

Risque de généralisation

La quantité pertinente est le **risque de généralisation** $R(f) = \mathbb{P}(f(X) \neq C)$. La loi jointe de (X, C) étant inconnue, on l'estime sur un ensemble de test tenu à l'écart du choix des variables, de l'estimation des transformations et de l'ajustement des paramètres.

Sur un test indépendant de 200 élèves dont 160 sont correctement classés, l'erreur observée vaut $40/200 = 0,20$. Elle estime le risque de généralisation : environ 20 % des futurs élèves comparables seraient mal classés. L'estimation porte sa propre variabilité, un autre échantillon de 200 élèves donnant un résultat voisin et différent.

1.1 Décision et calibration probabiliste

Désigner la bonne maison et annoncer une probabilité juste sont deux exigences distinctes. La première ne demande que de placer le bon candidat en tête. La seconde demande qu'un nombre annoncé se vérifie sur le long terme, ce qui est nettement plus difficile à obtenir.

Attribuer une maison à un nouvel élève est une décision. Estimer une probabilité d'appartenance **calibrée** est une tâche plus exigeante : parmi les élèves auxquels le modèle attribue une probabilité proche de 0,8, environ 80 % appartiennent effectivement à la classe. OvR répond à la première question ; la seconde demande les précautions du chapitre 6.

Supposons que le modèle attribue une probabilité comprise entre 0,75 et 0,85 à Ravenclaw pour 50 élèves. Si 39 sont réellement Ravenclaw, la fréquence observée vaut $39/50 = 0,78$ et confirme l'annonce. Si 20 le sont, elle tombe à 0,40 : le classement par maximum des scores reste utilisable, et la lecture probabiliste de ces nombres demande une recalibration préalable.

Objets définis par le problème

Le jeu étiqueté fournit une matrice de caractéristiques et un vecteur de maisons. L'apprentissage produit des paramètres. La prédiction transforme une ligne de caractéristiques en une maison. L'évaluation compare le vecteur des maisons prédites au vecteur des maisons réelles. Ces objets gardent des rôles séparés dans toute la chaîne.

Chapitre 2

Données, préparation et protocole expérimental

Les notes du fichier ne sont pas utilisables telles quelles : des cases sont vides, et deux colonnes voisines se mesurent parfois sur des échelles sans rapport l'une avec l'autre. Trois opérations les rendent calculables — boucher les trous, ramener les colonnes à une échelle commune, ajouter une colonne constante — sous une règle unique : chacune s'apprend sur les seules lignes d'apprentissage, puis s'applique inchangée à toutes les autres.

2.1 Matrice de design

Une matrice de design est le tableau numérique reçu par le modèle. Chaque ligne décrit un élève, chaque colonne une caractéristique. Une colonne supplémentaire remplie de 1 ajoute le même terme constant au score de tous les élèves.

Après préparation, on ajoute une coordonnée constante à chaque observation :

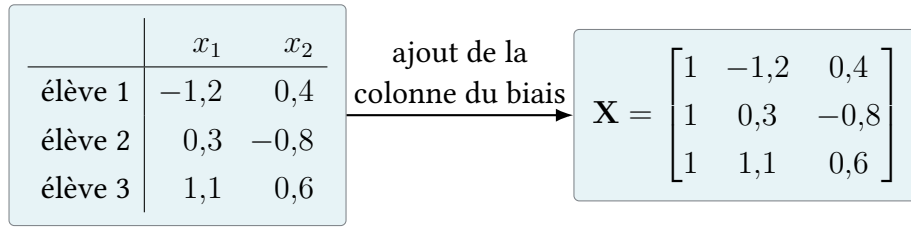
$$\tilde{\mathbf{x}}_i = (1, x_{i1}, \dots, x_{in})^\top \in \mathbb{R}^{n+1}.$$

La matrice de design empile les observations par lignes,

$$\mathbf{X} = \begin{bmatrix} \tilde{\mathbf{x}}_1^\top \\ \vdots \\ \tilde{\mathbf{x}}_m^\top \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}.$$

Cette convention fixe toutes les dimensions ultérieures : les paramètres sont des vecteurs colonnes de $\mathbb{R}^{n+1}$ et $\mathbf{X}\boldsymbol{\theta} \in \mathbb{R}^m$.

Sur trois élèves décrits par deux notes standardisées, la construction devient par exemple



une ligne = une observation ; une colonne = une caractéristique
la première colonne, constante, est associée à θ_0

$$\underbrace{\begin{bmatrix} 1 & -1,2 & 0,4 \\ 1 & 0,3 & -0,8 \\ 1 & 1,1 & 0,6 \end{bmatrix}}_{\mathbf{X} \ (3 \times 3)} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}}_{\boldsymbol{\theta} \ (3 \times 1)} = \underbrace{\begin{bmatrix} \theta_0 - 1,2\theta_1 + 0,4\theta_2 \\ \theta_0 + 0,3\theta_1 - 0,8\theta_2 \\ \theta_0 + 1,1\theta_1 + 0,6\theta_2 \end{bmatrix}}_{\mathbf{X}\boldsymbol{\theta} = (z_1, z_2, z_3)^\top \ (3 \times 1)}$$

FIGURE 2.1 – Construction concrète d’une matrice de design. La colonne de 1 multiplie θ_0 dans chaque score et encode ainsi le terme constant. Ici $\mathbf{X} \in \mathbb{R}^{3 \times 3}$, $\boldsymbol{\theta} \in \mathbb{R}^3$ et $\mathbf{X}\boldsymbol{\theta} \in \mathbb{R}^3$.

Sous-procédure – Former la matrice de design

Entrée : m observations déjà préparées, chacune décrite par les mêmes n caractéristiques dans un ordre fixé. **Sortie :** une matrice $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$.

1. écrire les n valeurs de l’observation i sur la ligne i ;
2. ajouter à gauche une colonne de 1, associée au biais θ_0 ;
3. enregistrer l’ordre des colonnes, par exemple (1, Astronomy, Herbology, ...) ;
4. vérifier que chaque ligne contient $n + 1$ nombres et que $\mathbf{X}\boldsymbol{\theta}$ produit exactement m scores.

Avec deux caractéristiques, le rôle de la constante apparaît dans l’équation de la frontière. Le score $\theta_1 x_1 + \theta_2 x_2$ place la frontière sur une droite passant par l’origine. Le score $\theta_0 + \theta_1 x_1 + \theta_2 x_2$ la place sur $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, qui se déplace parallèlement à elle-même quand θ_0 varie. Le biais est le paramètre qui règle cette translation.

2.2 Valeurs manquantes et standardisation

Le calcul exige un nombre dans chaque case et des échelles comparables. Imputer consiste à définir la valeur utilisée lorsqu’une mesure manque. **Standardiser** consiste à exprimer ensuite chaque mesure par rapport à la moyenne et à l’écart-type calculés sur l’apprentissage.

Les opérations de la régression logistique portent sur des nombres. Chaque case vide demande donc une règle explicite, par exemple l’imputation par la **médiane**, et la valeur d’imputation de chaque variable rejoint l’état conservé du modèle. La suppression d’une ligne relève du même principe : un choix, énoncé et documenté.

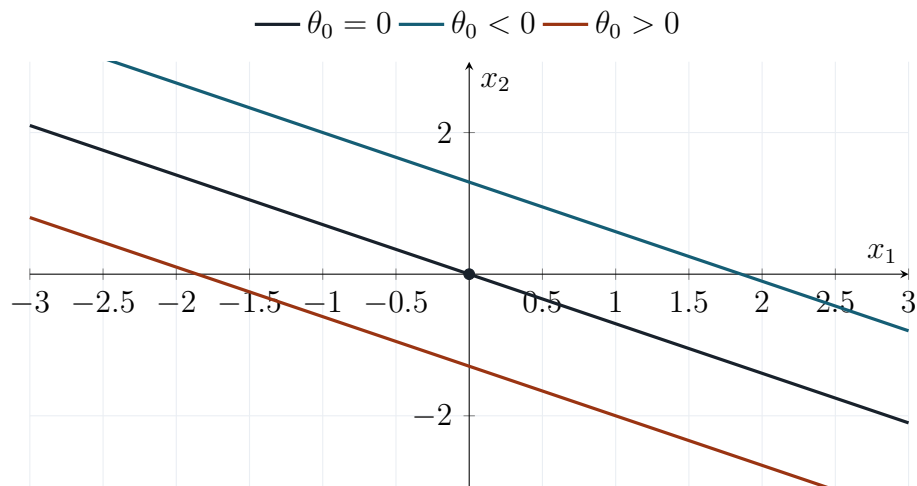


FIGURE 2.2 – Effet du biais pour des poids θ_1 et θ_2 fixés. Modifier θ_0 translate la frontière parallèlement à elle-même ; sans biais, elle est contrainte de passer par l’origine.

Exemple : les valeurs d’apprentissage d’un cours sont $\{4, 7, 9, 12, 100\}$. La médiane vaut 9 et remplace une note manquante. La moyenne vaudrait 26,4, tirée vers le haut par la seule valeur 100. La médiane offre donc ici une robustesse aux valeurs extrêmes ; en contrepartie, l’imputation contracte la dispersion et absorbe un éventuel mécanisme informatif derrière l’absence.

L’ordre retenu dans ce document – imputer, puis estimer μ_j et s_j sur la colonne complétée – applique une règle unique à l’apprentissage, à la validation et à toute observation future. Il contracte s_j d’autant que la proportion d’imputations est grande. Estimer μ_j et s_j sur les seules valeurs observées est l’autre convention défendable. Le point déterminant est l’enregistrement du choix et son application identique à chaque étape.

valeur observée		valeur préparée
4		4
7		7
manquante	$\xrightarrow{\text{médiane}=9}$	9
12		12
100		100

FIGURE 2.3 – Imputation médiane sur une variable. La case colorée porte une valeur calculée, d’un statut distinct des quatre mesures qui l’entourent. La médiane provient des seules lignes d’apprentissage.

Pour une note brute r_j , on estime sur l’apprentissage

$$\mu_j = \frac{1}{m_{\text{tr}}} \sum_{i \in \mathcal{I}_{\text{tr}}} r_{ij}, \quad s_j = \sqrt{\frac{1}{m_{\text{tr}}} \sum_{i \in \mathcal{I}_{\text{tr}}} (r_{ij} - \mu_j)^2}, \quad x_j = \frac{r_j - \mu_j}{s_j}.$$

Le diviseur de l'écart-type, m_{tr} ou $m_{tr} - 1$, change le résultat de moins d'un pour mille sur 1 600 lignes ; le même reste employé à l'entraînement et à la prédiction. Un s_j nul signale une colonne constante sur l'apprentissage, que le programme retire de la liste des variables.

La standardisation reparamètre le modèle et conserve la famille des frontières linéaires. Elle améliore le conditionnement numérique de l'optimisation et rend une éventuelle pénalisation comparable entre variables [1, sec. 4.6.3].

Le conditionnement mesure la sensibilité du calcul aux directions de l'espace des paramètres. Une variable qui varie autour de 10^3 et une autre autour de 10^{-2} répondent à un même déplacement de leurs coefficients par des variations de score dans un rapport de 10^5 . Les courbes de niveau du coût s'allongent d'autant, et la descente zigzague ou progresse par pas très courts. La figure 5.2 en donne la représentation géométrique. La standardisation ramène les échelles à 1 et conserve l'ordre des observations dans chaque colonne.

Exemple de standardisation

Supposons que la note Astronomy ait, sur l'apprentissage, une moyenne $\mu = 20$ et un écart-type $s = 100$. Les notes brutes -180 , 20 et 170 deviennent respectivement -2 , 0 et $1,5$. La transformation conserve leur ordre et leurs positions relatives, et fixe une origine et une échelle communes. Une valeur standardisée de $1,5$ se lit « un écart-type et demi au-dessus de la moyenne d'apprentissage », dans l'unité propre à cette colonne.

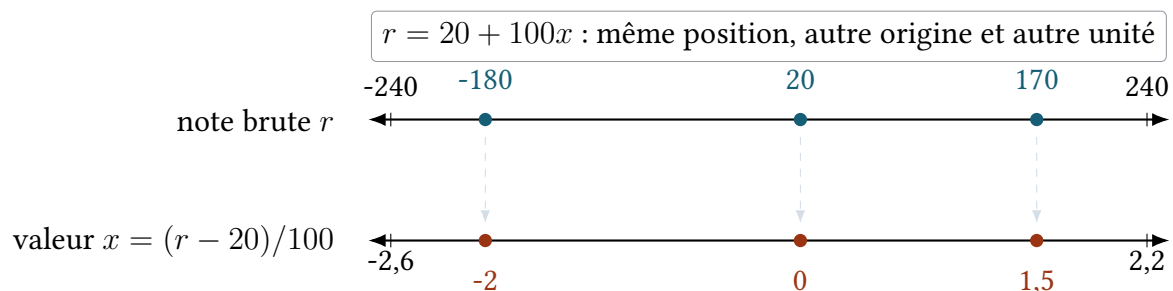


FIGURE 2.4 – Correspondance complète entre l'axe des notes brutes et l'axe standardisé. Chaque flèche verticale relie deux écritures de la même observation : $-180 \leftrightarrow -2$, $20 \leftrightarrow 0$ et $170 \leftrightarrow 1,5$.

Fuite de données

Calculer μ_j , s_j , une médiane d'imputation ou choisir des variables à partir de l'ensemble complet transmet de l'information du test vers l'apprentissage, et rend l'évaluation optimiste. La partition précède donc toute transformation apprise des données.

Sous-procédure — Préparer une caractéristique numérique

Entrée : les valeurs brutes r_{ij} de la caractéristique j sur les lignes d'apprentissage. **Sortie :** la valeur d'imputation, μ_j , s_j et les valeurs transformées x_{ij} .

1. calculer la médiane des valeurs présentes et l'utiliser pour remplacer les valeurs manquantes ;
2. calculer ensuite μ_j et s_j sur cette colonne d'apprentissage complétée ;
3. transformer chaque valeur par $x_{ij} = (r_{ij} - \mu_j)/s_j$; par exemple, $r = 170$, $\mu = 20$, $s = 100$ donnent $x = 1,5$;
4. conserver médiane, μ_j et s_j , et les réappliquer telles quelles à la validation, au test et à toute observation future ;
5. si $s_j = 0$, signaler une colonne constante et la retirer du calcul.

2.3 Partition et déséquilibre des classes

La partition produit deux listes d'indices disjointes : les indices d'apprentissage $\mathcal{I}_{\text{tr}}$ et les indices de test $\mathcal{I}_{\text{te}}$. Une ligne appartient à une seule liste. Toutes les transformations et tous les coefficients sont estimés depuis la première ; la seconde reste intacte jusqu'à l'évaluation finale.

Les deux listes vérifient

$$\mathcal{I}_{\text{tr}} \cap \mathcal{I}_{\text{te}} = \emptyset, \quad \mathcal{I}_{\text{tr}} \cup \mathcal{I}_{\text{te}} = \{1, \dots, m\}.$$

Pour construire une partition **stratifiée**, on regroupe d'abord les indices par classe. Dans chaque groupe, on mélange les indices avec une graine fixée, puis on affecte la proportion prévue à l'apprentissage et le reste au test. Les morceaux obtenus sont enfin réunis dans les deux listes. L'opération porte sur les indices et laisse les données numériques en place.

L'état à conserver tient donc en trois objets : la graine et les deux listes d'indices. Moyennes, écarts-types et valeurs d'imputation viennent ensuite des seules lignes désignées par $\mathcal{I}_{\text{tr}}$, puis s'appliquent telles quelles aux lignes de $\mathcal{I}_{\text{te}}$.

2.4 Préservation des effectifs par classe

Un tirage au sort global peut vider une classe rare de l'un des deux sous-ensembles. Stratifier consiste à tirer séparément à l'intérieur de chaque classe : les proportions d'origine se retrouvent alors des deux côtés.

Avec 1 000 élèves, dont 900 majoritaires et 100 dans la classe étudiée, une répartition 80 %/20 % affecte séparément 80 % de chaque classe à l'apprentissage. On obtient $720 + 80 = 800$ lignes d'apprentissage et $180 + 20 = 200$ lignes de test. Le déséquilibre 90 %/10 % se retrouve à l'identique dans les deux sous-ensembles.

classe	ensemble disponible	apprentissage (80 %)	test (20 %)
majoritaire	900	720	180
classe étudiée	100	80	20
total	1 000	800	200

$$900 = 720 + 180$$

$$100 = 80 + 20$$

FIGURE 2.5 – Effectifs produits par une partition stratifiée 80 %/20 %.

Un **hyperparamètre** est une quantité que le protocole fixe avant l’ajustement. Les coefficients θ sont des paramètres appris par le gradient ; le taux α et la pénalisation λ sont des hyperparamètres. Si trois valeurs de λ donnent sur validation des exactitudes 0,78, 0,83 et 0,80, la deuxième est retenue, puis le test intervient une seule fois. La validation croisée du chapitre 7 étend ce principe en faisant tourner le rôle de validation sur plusieurs blocs.

Quantités fixées après la préparation

La préparation produit les listes d’indices d’apprentissage et de test, une valeur d’imputation par caractéristique, une moyenne et un écart-type par caractéristique, puis les matrices numériques transformées. Ces quantités sont estimées une fois sur l’apprentissage et réutilisées sans modification pour la validation, le test et toute observation future.

Sous-procédure — Construire une partition stratifiée

Entrées : les numéros de lignes $1, \dots, m$, leur classe c_i , une proportion d’apprentissage et une **graine**. La graine est un entier qui initialise le générateur pseudo-aléatoire : reprendre le même entier reproduit le même mélange. **Sorties** : deux listes d’indices $\mathcal{I}_{\text{tr}}$ et $\mathcal{I}_{\text{te}}$.

1. former une liste de numéros de lignes par classe ; par exemple, une liste contient les 100 indices de la classe étudiée ;
2. mélanger chaque liste après initialisation avec la graine enregistrée ;
3. pour une proportion 80 %, placer les 80 premiers indices de cette liste dans $\mathcal{I}_{\text{tr}}$ et les 20 autres dans $\mathcal{I}_{\text{te}}$; répéter pour chaque classe ;
4. concaténer les morceaux obtenus : dans l’exemple complet, les deux listes contiennent respectivement 800 et 200 indices ;
5. vérifier qu’aucun indice n’apparaît deux fois, qu’aucun ne manque et que les effectifs par classe correspondent à la table 2.5 ;
6. enregistrer définitivement graine et listes avant de calculer médianes, moyennes ou écarts-types.

Chapitre 3

Modèle logistique binaire

Le cas binaire pose une question à deux réponses, par exemple « Ravenclaw ou non-Ravenclaw ? ». Les caractéristiques produisent d'abord un score réel, qui peut être négatif ou positif. La **sigmoïde** convertit ce score en un nombre entre 0 et 1 interprété comme une probabilité conditionnelle.

Considérons provisoirement des étiquettes $y_i \in \{0, 1\}$. On pose

$$z_{\theta}(\mathbf{x}) = \boldsymbol{\theta}^{\top} \tilde{\mathbf{x}}, \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad p_{\theta}(\mathbf{x}) = \sigma(z_{\theta}(\mathbf{x})).$$

Cette paramétrisation définit la régression logistique binaire [1, 2, sec. 4.4].

Pour un ensemble de m observations, le même calcul s'écrit en une opération : $\mathbf{X}\boldsymbol{\theta}$ produit un vecteur de m scores, puis la sigmoïde appliquée composante par composante produit un vecteur de m probabilités. L'apprentissage fait varier le seul vecteur $\boldsymbol{\theta}$; $\mathbf{X}$ garde sa valeur pendant les itérations.

3.1 Interprétation par les log-cotes

Les **cotes** comparent la probabilité de l'événement à celle de son contraire. Une probabilité de 0,75 correspond à des cotes $0,75/0,25 = 3$: l'événement est trois fois plus probable que son contraire. Le logarithme transforme les multiplications de cotes en additions, compatibles avec un score linéaire.

La fonction logistique est équivalente au modèle linéaire

$$\log \frac{\mathbb{P}(Y = 1 \mid X = \mathbf{x})}{\mathbb{P}(Y = 0 \mid X = \mathbf{x})} = \boldsymbol{\theta}^{\top} \tilde{\mathbf{x}}.$$

Cette égalité est la formulation en **log-cotes** du modèle logistique [1, équations 4.17–4.18].

Le modèle suppose que les log-cotes sont une fonction affine des caractéristiques. La transformation inverse est

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Le calcul reçoit donc un score réel $z = \boldsymbol{\theta}^\top \tilde{\mathbf{x}}$ et produit une probabilité comprise entre 0 et 1. Les étapes algébriques de l'inversion sont données à l'annexe A.1. À caractéristiques égales par ailleurs, augmenter x_j d'une unité multiplie les *cotes* de la classe positive par e^{θ_j} . Après standardisation, cette unité vaut un écart-type de la variable brute. Le facteur constant e^{θ_j} porte sur les cotes conditionnelles ; l'effet correspondant sur la probabilité dépend du niveau de départ.

Prenons $\theta_j = \log 2 \approx 0,693$, donc $e^{\theta_j} = 2$: une unité de x_j double les cotes. Partant d'une probabilité de 0,20, les cotes valent $0,20/0,80 = 0,25$, passent à 0,50, et donnent une probabilité $0,50/(1 + 0,50) = 1/3$. Le rapport de chances a doublé, la probabilité est passée de 0,20 à 0,33.

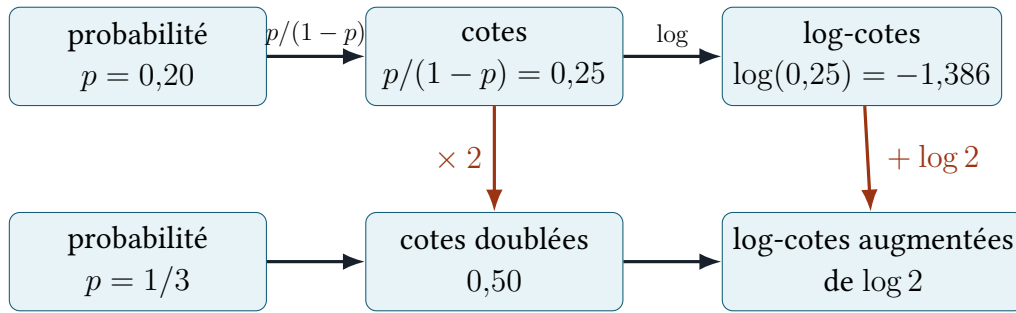


FIGURE 3.1 – Passage entre probabilité, cotes et log-cotes. Ajouter $\log 2$ au score double les cotes et porte ici la probabilité de 0,20 à $1/3$.

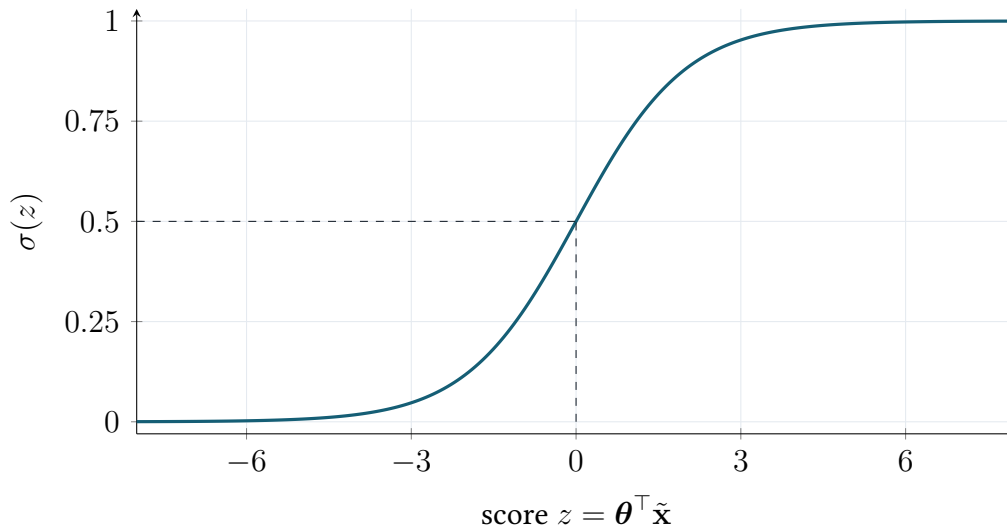


FIGURE 3.2 – La logistique est strictement croissante, vérifie $\sigma(-z) = 1 - \sigma(z)$ et $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Le seuil de probabilité $1/2$ correspond donc au score nul.

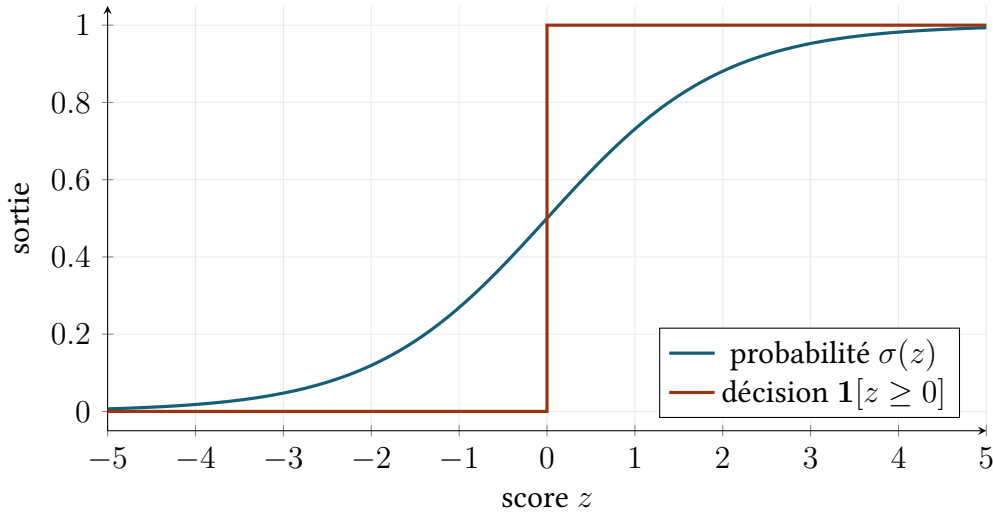


FIGURE 3.3 – Deux sorties du même score. La sigmoïde décrit une transition continue et conserve le degré de confiance ; le seuil produit une étiquette binaire, constante de part et d’autre de $z = 0$.

Sous-procédure — Calculer une sortie binaire

Entrées : une ligne $\tilde{\mathbf{x}} \in \mathbb{R}^{n+1}$ et les paramètres $\boldsymbol{\theta} \in \mathbb{R}^{n+1}$. **Sorties :** un score réel z et une probabilité p .

1. vérifier que $\tilde{\mathbf{x}}$ et $\boldsymbol{\theta}$ ont la même longueur et le même ordre de colonnes ;
2. calculer $z = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$;
3. calculer $p = 1/(1 + e^{-z})$ avec la forme stable du chapitre 5 ;
4. vérifier $0 < p < 1$ et conserver z séparément de p : OvR compare les scores.

3.2 Frontière de décision

La probabilité varie continûment, la décision demande une classe. Avec un seuil de 0,5, la frontière réunit les observations placées à égale probabilité entre les deux réponses. En deux dimensions elle est une droite, au-delà un hyperplan.

Avec des coûts d’erreur symétriques, la règle de Bayes estimée est

$$\hat{y}(\mathbf{x}) = \mathbf{1}[p_{\boldsymbol{\theta}}(\mathbf{x}) \geq 1/2] = \mathbf{1}[\boldsymbol{\theta}^\top \tilde{\mathbf{x}} \geq 0].$$

La règle générale consiste à choisir la classe de probabilité conditionnelle maximale ; le seuil $1/2$ en est le cas binaire à coûts symétriques [1, sec. 2.4]. La frontière $\{\mathbf{x} : \boldsymbol{\theta}^\top \tilde{\mathbf{x}} = 0\}$ est un hyperplan. La transformation logistique borne la sortie entre 0 et 1 et laisse la frontière linéaire dans les variables fournies au modèle.

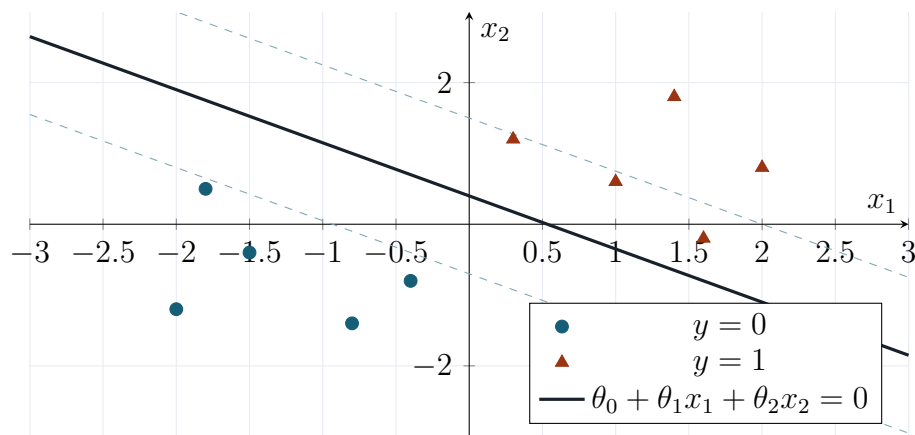


FIGURE 3.4 – Exemple à deux caractéristiques. La droite pleine est la frontière $p = 0,5$; les droites pointillées sont des lignes d'égale probabilité. Le score étant affine, elles restent parallèles. Les points sont construits pour la figure.

Sous-procédure — Appliquer une décision binaire

Entrée : une probabilité p et un seuil τ fixé avant le test. **Sortie :** une étiquette $\hat{y} \in \{0, 1\}$.

1. avec des coûts symétriques, choisir $\tau = 0,5$;
2. produire $\hat{y} = 1$ si $p \geq \tau$, sinon $\hat{y} = 0$;
3. exemple : $p = 0,71$ donne 1, tandis que $p = 0,23$ donne 0;
4. si un autre seuil est nécessaire, le sélectionner sur validation et enregistrer sa valeur avant l'évaluation du test.

Quantités produites par un modèle binaire

Une ligne préparée $\tilde{\mathbf{x}}$ et un vecteur de paramètres $\boldsymbol{\theta}$ produisent un score $z = \boldsymbol{\theta}^\top \tilde{\mathbf{x}}$, puis une probabilité $p = \sigma(z)$, puis une décision binaire par comparaison au seuil. L'apprentissage fixe $\boldsymbol{\theta}$; la prédiction le lit et l'applique.

Chapitre 4

Estimation par maximum de vraisemblance

L'apprentissage compare les probabilités annoncées aux réponses observées. Un paramétrage reçoit une vraisemblance élevée lorsqu'il attribue une forte probabilité aux étiquettes présentes dans le fichier. La log-perte est cette même comparaison écrite comme un coût à minimiser.

On suppose les observations indépendantes conditionnellement à leurs variables explicatives et

$$Y_i \mid X_i = \mathbf{x}_i \sim \text{Bernoulli}(p_i), \quad p_i = \sigma(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i).$$

Ce modèle de [Bernoulli](#) conditionnel et son estimation par [maximum de vraisemblance](#) sont la construction standard de la régression logistique [1, 2, sec. 4.4.1].

À une valeur donnée de $\boldsymbol{\theta}$ correspondent donc trois objets successifs : le vecteur des scores $\mathbf{X}\boldsymbol{\theta}$, le vecteur des probabilités $\mathbf{p}$ et un coût scalaire $J(\boldsymbol{\theta})$ qui résume l'accord avec toutes les cibles. L'apprentissage réévalue ces objets après chaque modification de $\boldsymbol{\theta}$; les observations et leurs cibles restent fixes.

La masse de probabilité d'une [Bernoulli](#) peut s'écrire sous une forme unique pour les deux valeurs possibles de y_i :

$$\mathbb{P}(Y_i = y_i \mid X_i = \mathbf{x}_i; \boldsymbol{\theta}) = p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Si $y_i = 1$, cette expression vaut p_i ; si $y_i = 0$, elle vaut $1 - p_i$. Sous l'hypothèse d'indépendance conditionnelle des observations, la probabilité jointe des réponses observées est le produit de ces contributions. Ce passage produit la vraisemblance ci-dessous.

Cette indépendance concerne les élèves une fois leurs caractéristiques fixées. Elle permet d'écrire, pour deux observations,

$$\begin{aligned} \mathbb{P}(Y_1 = y_1, Y_2 = y_2 \mid X_1 = \mathbf{x}_1, X_2 = \mathbf{x}_2; \boldsymbol{\theta}) \\ = \mathbb{P}(Y_1 = y_1 \mid X_1 = \mathbf{x}_1; \boldsymbol{\theta}) \mathbb{P}(Y_2 = y_2 \mid X_2 = \mathbf{x}_2; \boldsymbol{\theta}). \end{aligned}$$

Des lignes dupliquées pour un même élève, ou des jumeaux dont les réponses restent liées une fois les variables connues, la mettraient en défaut. Le produit de vraisemblance traduit donc une hypothèse sur la manière dont les données ont été recueillies. La vraisemblance et sa log-vraisemblance sont

$$L(\boldsymbol{\theta}) = \prod_{i=1}^m p_i^{y_i} (1 - p_i)^{1-y_i},$$

$$\log L(\boldsymbol{\theta}) = \sum_{i=1}^m [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Maximiser L équivaut à minimiser la log-perte moyenne

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \log L(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (1)$$

L'équation (1) est l'opposé de la log-vraisemblance binomiale, également appelée **log-perte** ou entropie croisée binaire [1, équations 4.19–4.20]. Cette quantité est l'opposé du logarithme de la probabilité attribuée par le modèle aux étiquettes observées. Le sujet du projet écrit la même formule avec $h_\theta(x^i)$ à la place de p_i ; la section 9.11 donne la correspondance terme à terme.

i	y_i	p_i	probabilité attribuée au résultat observé	ℓ_i
1	1	0,90	0,90	$-\log(0,90) = 0,105$
2	0	0,20	$1 - 0,20 = 0,80$	$-\log(0,80) = 0,223$
3	1	0,10	0,10	$-\log(0,10) = 2,303$
perte moyenne J				0,877

TABLE 4.1 – Calcul de la log-perte sur trois observations. La troisième prédiction, fausse et assurée, domine la moyenne; c'est le comportement recherché par le maximum de vraisemblance.

Sous-procédure — Évaluer la log-perte

Entrées : m probabilités p_i et m cibles $y_i \in \{0, 1\}$. **Sortie :** un nombre $J(\boldsymbol{\theta})$; une valeur plus faible indique un meilleur accord avec les étiquettes observées.

1. calculer les scores z_i , puis les probabilités p_i ;
2. pour chaque ligne, prendre $a_i = p_i$ si $y_i = 1$, sinon $a_i = 1 - p_i$;
3. calculer $\ell_i = -\log(a_i)$; par exemple, $a_i = 0,9$ donne $\ell_i \approx 0,105$ et $a_i = 0,1$ donne $\ell_i \approx 2,303$;
4. calculer $J = m^{-1} \sum_i \ell_i$ et vérifier qu'il reste fini.

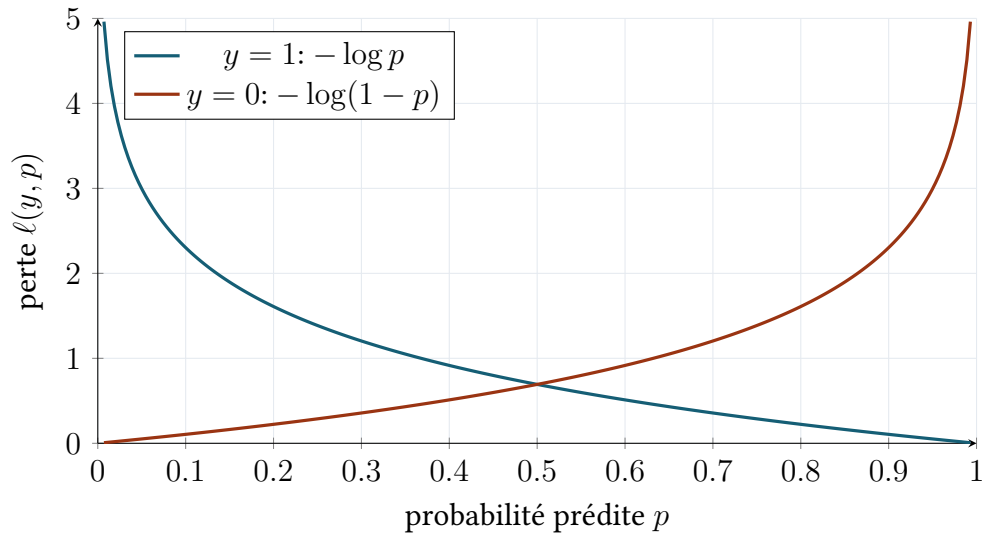


FIGURE 4.1 – Une prédiction correcte et assurée coûte presque zéro ; une prédiction erronée et assurée reçoit une pénalité non bornée.

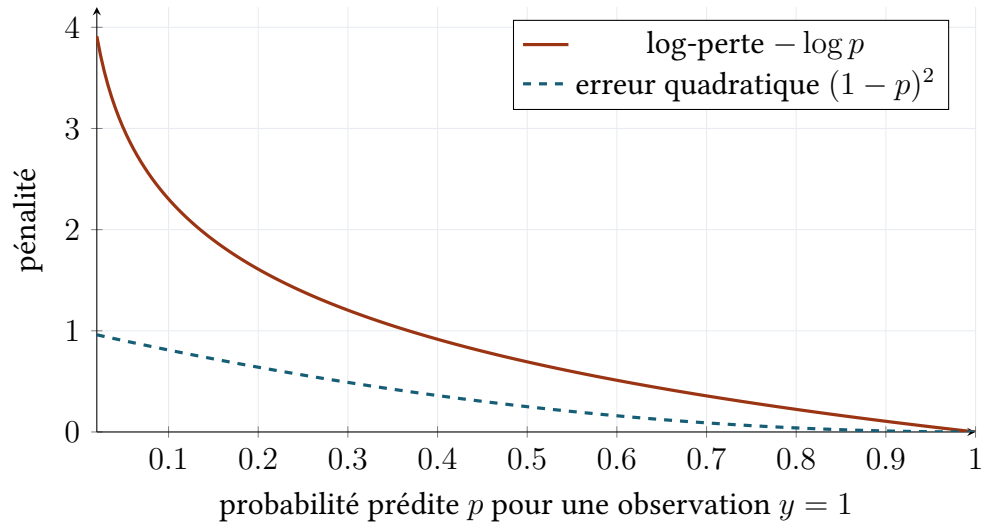


FIGURE 4.2 – Comparaison pour $y = 1$. Les deux pertes sont minimales en $p = 1$, mais la log-perte pénalise beaucoup plus fortement une probabilité presque nulle attribuée à l'événement réellement observé. Elle découle en outre directement de la vraisemblance de Bernoulli.

4.1 Gradient et correction des coefficients

Le gradient indique comment une petite modification de chaque coefficient ferait varier le coût. Chaque observation contribue selon son erreur $p_i - y_i$ et selon ses caractéristiques. La somme de ces contributions fournit la correction de tous les coefficients en une opération matricielle.

Pour chaque observation, le programme calcule d'abord l'écart $p_i - y_i$ entre la probabilité produite et la cible binaire. Il pondère ensuite cet écart par les caractéristiques de l'observation, puis additionne les contributions. Le calcul complet s'écrit

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^\top (\sigma(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}). \quad (2)$$

Cette expression est le **gradient** de la log-perte sous forme matricielle [1, équation 4.21]. Sa dérivation par la règle de chaîne figure à l'annexe A.2. Le gradient a bien la dimension $n + 1$: $\mathbf{X}^\top$ est de taille $(n + 1) \times m$ et le vecteur des résidus probabilistes est de taille m . Le sujet donne cette formule composante par composante ; la section 9.11 montre que les deux écritures effectuent le même calcul.

Le terme $(p_i - y_i)\tilde{\mathbf{x}}_i$ permet une lecture locale. Pour un positif $y_i = 1$ sous-estimé à $p_i = 0,1$, le résidu vaut $-0,9$: l'observation exerce une correction importante. Pour un positif déjà estimé à $0,99$, le résidu vaut seulement $-0,01$. Le gradient agrège ces corrections en tenant compte de la direction $\tilde{\mathbf{x}}_i$ de chaque observation.

Sous-procédure — Calculer le gradient

Entrées : $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$, $\mathbf{y} \in \{0, 1\}^m$ et $\boldsymbol{\theta} \in \mathbb{R}^{n+1}$. **Sortie :** $\mathbf{g} \in \mathbb{R}^{n+1}$, dont chaque composante indique la correction associée à un coefficient.

1. calculer $\mathbf{z} = \mathbf{X}\boldsymbol{\theta}$, de longueur m , puis $\mathbf{p} = \sigma(\mathbf{z})$;
2. calculer $\mathbf{r} = \mathbf{p} - \mathbf{y}$; un positif prédit à $0,1$ contribue par un écart $-0,9$;
3. calculer $\mathbf{g} = \mathbf{X}^\top \mathbf{r} / m$;
4. vérifier que $\mathbf{g}$ et $\boldsymbol{\theta}$ ont tous deux $n + 1$ composantes ;
5. avant l'entraînement complet, comparer quelques composantes à la différence finie de la figure 5.5.

4.2 Convexité, unicité et séparabilité

Une fonction convexe a la géométrie d'une cuvette : toute descente locale mène au minimum global. Une cuvette peut posséder un fond plat, et donc plusieurs jeux de paramètres équivalents. Avec des classes parfaitement séparées, ce fond s'étend à l'infini.

En notant $\mathbf{W} = \text{diag}(p_i(1 - p_i))$, la **Hessienne** vaut

$$\nabla^2 J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^\top \mathbf{W} \mathbf{X}.$$

Cette matrice $\mathbf{X}^\top \mathbf{W} \mathbf{X}$ est l'information observée, à un facteur de normalisation près [1, équations 4.22–4.25]. Les coefficients diagonaux $p_i(1 - p_i)$ sont positifs ou nuls ; la Hessienne est donc **positive semi-définie** et J est **convexe**. Tout minimum local est global. La preuve matricielle est donnée à l'annexe A.3. La stricte convexité, qui donne l'unicité du minimiseur, demande en plus que $\mathbf{X}$ soit de rang plein.

Convexité et stricte convexité se séparent ici. Une fonction convexe garantit que toute descente atteint le minimum global ; son fond peut néanmoins contenir plusieurs minimiseurs. Avec deux colonnes identiques dans $\mathbf{X}$, des variations opposées de leurs deux coefficients laissent tous les scores intacts : les prédictions sont uniques et le vecteur de paramètres appartient à une droite de solutions.

En une dimension, $J(\theta) = (\theta - 1)^2$ est strictement convexe : sa courbure $J''(\theta) = 2$ est positive et son unique minimum est $\theta = 1$. La fonction $J(\theta_1, \theta_2) = (\theta_1 + \theta_2 - 1)^2$ est seulement convexe : tous les couples vérifiant $\theta_1 + \theta_2 = 1$ minimisent le coût. Cette seconde situation reproduit ce qui se passe lorsque deux colonnes de la matrice de design apportent exactement la même information.

Séparation complète

Si un hyperplan sépare parfaitement les deux classes, la vraisemblance non pénalisée croît continûment quand $\|\boldsymbol{\theta}\| \rightarrow \infty$ et atteint sa borne supérieure à la limite. La convexité porte sur la forme de J ; l'existence d'un minimiseur fini est une propriété distincte, que la pénalisation L^2 rétablit [3].

Exemple : sur une caractéristique x , supposons que tous les négatifs vérifient $x < 0$ et tous les positifs $x > 0$. Le score $z = 10x$ classe déjà les observations avec assurance ; $z = 100x$ rapproche les probabilités de 0 et de 1 et augmente encore la vraisemblance. Chaque coefficient trouve donc un successeur meilleur, et la suite part vers $+\infty$. La pénalisation ajoute un coût croissant avec ce coefficient, ce qui ramène l'optimum à distance finie.

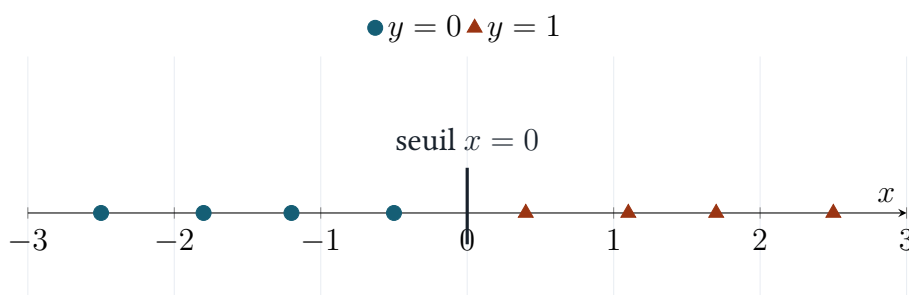


FIGURE 4.3 – Séparation complète sur une unique caractéristique x . Tous les points appartiennent au même axe : les cercles ($y = 0$) sont à gauche du seuil et les triangles ($y = 1$) à droite. Multiplier le coefficient conserve le seuil mais rend les probabilités toujours plus extrêmes.

Sous-procédure — Diagnostiquer l'existence d'une solution finie

Entrées : la matrice $\mathbf{X}$, les cibles $\mathbf{y}$ et l'historique des paramètres. **Sortie :** un diagnostic : ajustement ordinaire, redondance des colonnes ou séparation complète.

1. repérer deux colonnes identiques ou proportionnelles : leurs coefficients ne peuvent pas être identifiés séparément ;
2. vérifier si une frontière classe toutes les observations sans erreur ;
3. pendant l'ajustement, suivre $\|\boldsymbol{\theta}\|_2$ en plus de J : une norme qui croît continuellement tandis que J baisse signale une séparation probable ;
4. dans ce cas, activer une pénalisation L^2 et documenter le diagnostic.

4.3 Régularisation facultative

La régularisation arbitre entre deux objectifs : reproduire les observations et garder des coefficients de faible amplitude. Le paramètre λ fixe le poids du second. Une valeur nulle rend le modèle initial ; une valeur croissante contracte les coefficients vers zéro.

Une **pénalisation quadratique** donne

$$J_\lambda(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2, \quad \nabla J_\lambda = \nabla J + \frac{\lambda}{m} (0, \theta_1, \dots, \theta_n)^\top.$$

Cette forme correspond à la régression logistique pénalisée par une norme quadratique [1, sec. 4.4.4]. Le symbole

$$\|\mathbf{w}\|_2^2 = \theta_1^2 + \dots + \theta_n^2$$

est le carré de la **norme euclidienne** des coefficients hors biais. Ajouter cette quantité au coût introduit un second critère de jugement : entre deux solutions de log-pertes voisines, l'ajustement retient celle dont les coefficients sont les plus petits.

Le paramètre λ règle ce compromis. À $\lambda = 0$, le coût redevient le maximum de vraisemblance simple. Quand λ augmente, les coefficients convergent vers zéro et la prédiction se rapproche d'une constante gouvernée par le biais. La somme pénalisée court de $j = 1$ à n et laisse θ_0 libre : le modèle conserve ainsi la capacité de représenter la fréquence moyenne de la classe positive à toute intensité de pénalisation.

La réduction de variance signifie ici qu'une petite modification de l'échantillon d'apprentissage produit une petite modification des coefficients et des prédictions. Cette stabilité s'achète par un biais statistique supplémentaire : le modèle pénalisé s'ajuste volontairement moins étroitement aux données observées. Le meilleur compromis se détermine sur validation.

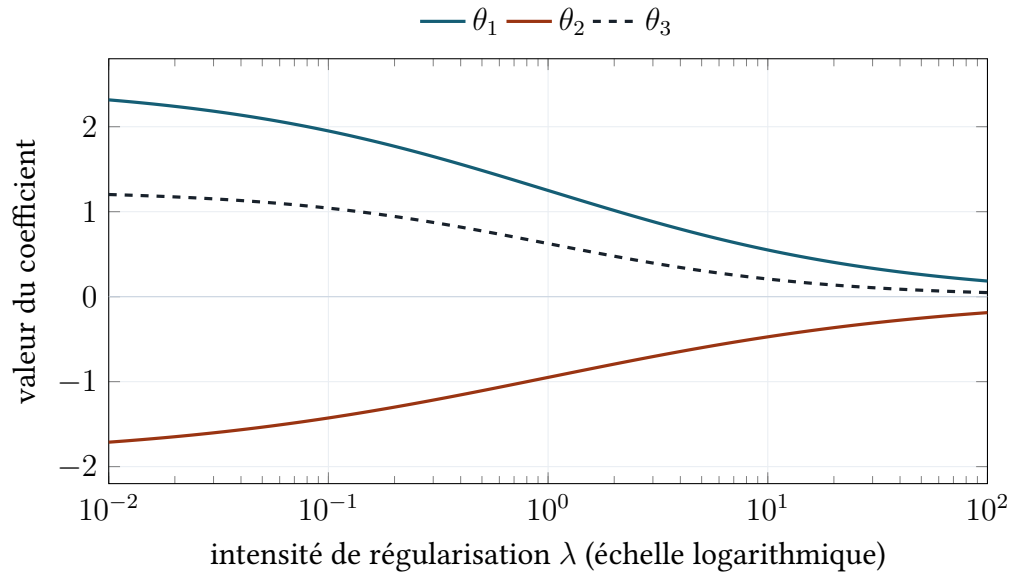


FIGURE 4.4 – Chemins de coefficients sous pénalisation L^2 : leur amplitude décroît lorsque λ augmente.

Plusieurs valeurs de λ sont donc comparées sur un ensemble de validation ou par validation croisée. Choisir λ d'après le test adapterait le modèle aux réponses du test, qui perdrait alors son statut d'évaluation indépendante. La figure 4.5 résume cette logique.

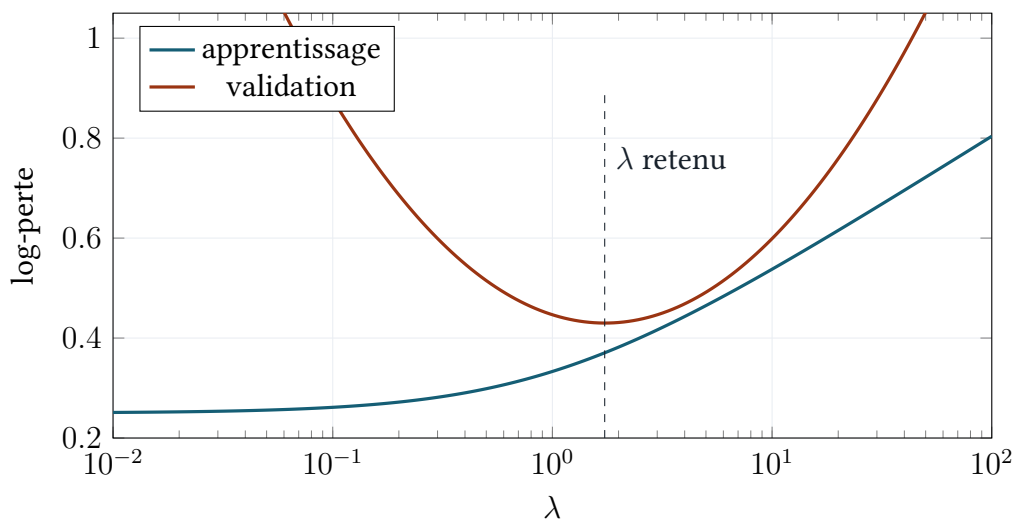


FIGURE 4.5 – Sélection de λ par minimisation de la log-perte de validation ; le test final reste exclu de cette sélection.

Sous-procédure — Sélectionner la pénalisation

Entrées : une grille explicite, par exemple $\lambda \in \{0, 10^{-2}, 10^{-1}, 1, 10\}$, et une partition de validation commune. **Sortie :** une valeur de λ choisie avant le test.

1. réinitialiser et ajuster un modèle pour chaque λ sur les mêmes lignes d'apprentissage ;
2. calculer pour chacun la log-perte sur les mêmes lignes de validation ;
3. construire le tableau $(\lambda, J_{\text{val}})$ et choisir la plus petite perte, avec une règle de départage fixée en cas d'égalité ;
4. enregistrer λ et les autres hyperparamètres retenus ;
5. appliquer ensuite le protocole final et consulter le test une seule fois.

Quantités recalculées pendant l'ajustement

À chaque itération, les paramètres courants produisent les scores, les probabilités, la log-perte moyenne et le gradient. La log-perte mesure l'état de l'ajustement ; le gradient produit la correction suivante. La régularisation ajoute sa contribution au coût et au gradient avant cette correction.

Chapitre 5

Optimisation et calcul numérique

La fonction de coût associe un nombre à chaque choix des coefficients. Optimiser consiste à déplacer progressivement ces coefficients vers une zone où ce nombre est plus faible. Le gradient donne la direction de plus forte hausse ; la descente suit donc la direction opposée, avec une longueur de pas réglée par α .

La **descente de gradient** simultanée produit la suite

$$\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \alpha \nabla J(\boldsymbol{\theta}^{[t]}), \quad \alpha > 0.$$

Cette itération est la descente par lot appliquée à la log-vraisemblance négative [4, chap. 8].

Le gradient donne la direction locale de hausse du coût, son opposé la direction locale de baisse. Le développement au premier ordre de l'annexe A.4 l'établit. La propriété vaut au voisinage du point courant, et un pas trop grand sort du domaine où cette approximation décrit la fonction.

La mise à jour est *simultanée* : les $n + 1$ composantes du nouveau vecteur se calculent toutes à partir du même ancien vecteur. Un taux élevé fait remonter le coût ou diverger ; un taux très petit allonge la convergence.

Un arrêt robuste combine un nombre maximal d'itérations avec, par exemple,

$$\frac{|J^{(t+1)} - J^{(t)}|}{\max(1, |J^{(t)}|)} < \varepsilon \quad \text{ou} \quad \|\nabla J(\boldsymbol{\theta}^{[t]})\|_2 < \varepsilon.$$

Le plafond d'itérations borne le temps de calcul ; le test sur J ou sur le gradient atteste la convergence.

Trois diagnostics coexistent, chacun sur sa propre question. Une baisse du coût dit que l'algorithme avance. Une petite norme du gradient dit qu'il approche un point stationnaire. Une bonne performance de validation dit que ce point produit un modèle utile hors échantillon. Chacun se vérifie séparément.

Un **point stationnaire** est un point où le gradient s'annule. Pour $J(\theta) = \theta^2$, le point 0 est stationnaire et donne le minimum. Pour $J(\theta) = \theta^3$, le point 0 est stationnaire alors que la fonction décroît à sa gauche et croît à sa droite : c'est un point d'inflexion. Dans le problème

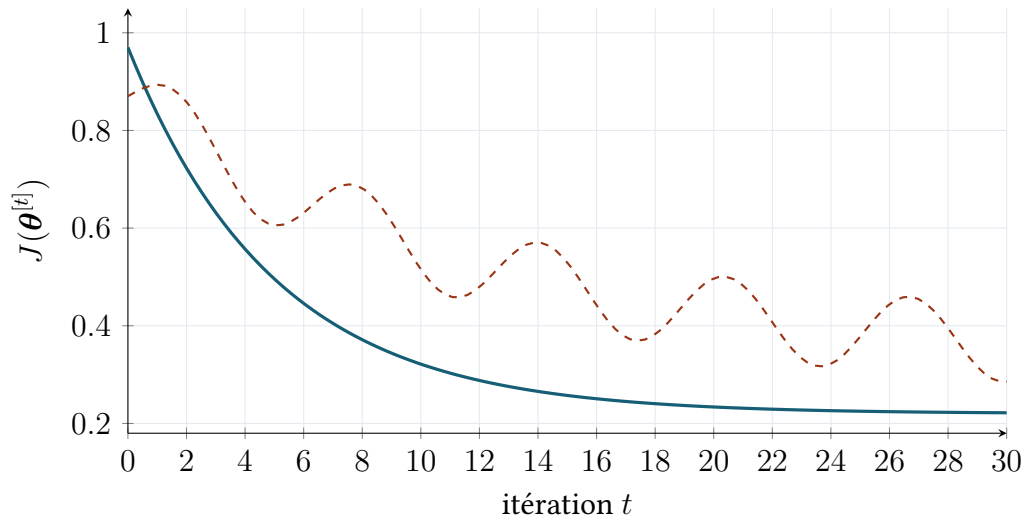


FIGURE 5.1 – Deux allures du coût au fil des itérations : décroissance stable en trait plein, oscillations d’un pas trop agressif en pointillé. Courbes construites pour la figure ; le tracé du coût réel rejoint l’archive de l’expérience.

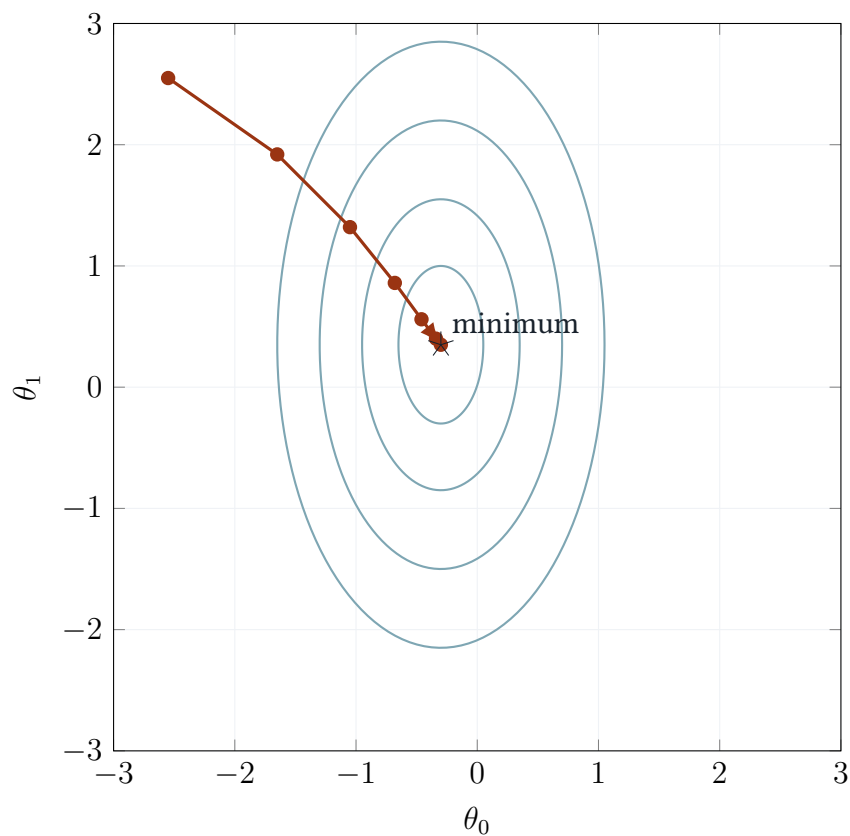


FIGURE 5.2 – Courbes de niveau schématiques de $J(\theta_0, \theta_1)$ et trajectoire de descente. L’allongement des ellipses traduit un mauvais conditionnement : un même taux d’apprentissage progresse vite dans une direction et lentement dans l’autre. Les points successifs aboutissent ici au centre des courbes, le minimum. La standardisation tend à réduire cet effet d’allongement.

logistique, la convexité établie au chapitre précédent fait de tout point stationnaire fini un minimum global.

5.1 Influence du taux d'apprentissage dans le cas scalaire

Une fonction d'une seule variable suffit à montrer ce que règle α : la longueur du pas. Trop court, il exige un grand nombre de tours ; trop long, il fait sauter le paramètre par-dessus le minimum, d'un côté puis de l'autre, sans jamais s'en rapprocher.

La fonction $J(\theta) = (\theta - 2)^2$ mesure ici la distance quadratique au minimum situé en $\theta = 2$. À chaque tour, le programme évalue la pente $J'(\theta) = 2(\theta - 2)$, la multiplie par α , puis soustrait cette correction à la valeur courante. Depuis $\theta^{[0]} = -2$, le pas $\alpha = 0,25$ rapproche successivement le paramètre du minimum, tandis que $\alpha = 1$ le fait passer alternativement d'un côté à l'autre sans réduire l'écart.

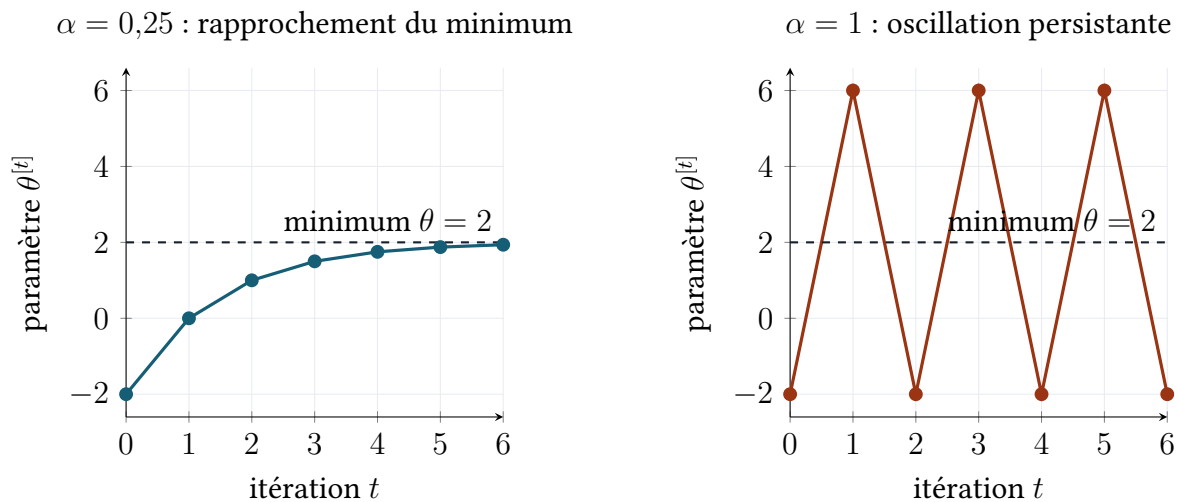


FIGURE 5.3 – Valeur du paramètre après chaque mise à jour. Les segments relient deux itérations successives et marquent l'ordre des calculs.

Dans le modèle complet, la même séquence s'applique à un vecteur : calculer les scores, les probabilités, le gradient, puis mettre à jour simultanément tous les coefficients. Le suivi de J permet de détecter une décroissance, une oscillation ou une divergence.

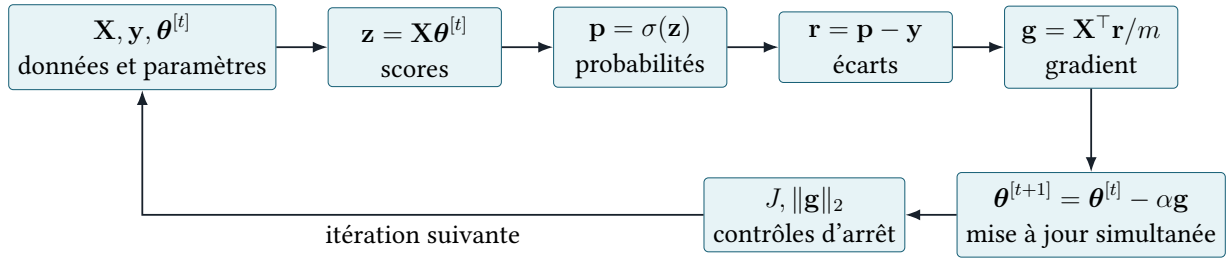


FIGURE 5.4 – Cycle de calcul d’une itération de descente de gradient par lot. Chaque flèche correspond à une transformation nécessaire ; aucun réajustement des données préparées n’intervient dans cette boucle.

Sous-procédure — Exécuter une itération d’ajustement

Entrées : $\mathbf{X}$, $\mathbf{y}$, les paramètres courants $\boldsymbol{\theta}^{[t]}$ et le taux α . **Sorties :** $\boldsymbol{\theta}^{[t+1]}$, le coût $J^{[t]}$ et la norme du gradient $\|\mathbf{g}^{[t]}\|_2$.

1. calculer $\mathbf{z} = \mathbf{X}\boldsymbol{\theta}^{[t]}$, $\mathbf{p} = \sigma(\mathbf{z})$, $J^{[t]}$ et $\mathbf{g}^{[t]} = \mathbf{X}^\top(\mathbf{p} - \mathbf{y})/m$;
2. calculer toutes les composantes de $\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \alpha\mathbf{g}^{[t]}$ depuis le même ancien vecteur ;
3. ajouter $J^{[t]}$ et $\|\mathbf{g}^{[t]}\|_2$ aux historiques ;
4. arrêter si la tolérance ou le nombre maximal d’itérations est atteint ;
5. sinon remplacer t par $t + 1$; si J augmente durablement ou devient non fini, interrompre et revoir α ou la stabilité des calculs.

5.2 Formules numériquement stables

Deux formules mathématiquement égales se comportent différemment sur un ordinateur, dont les nombres ont une taille et une précision finies. Les formes stables maintiennent les exponentielles dans le domaine représentable et travaillent sur le score plutôt que sur une probabilité arrondie à 0 ou 1.

Pour un score très négatif, e^{-z} dépasse la capacité du format flottant. Une sigmoïde stable utilise donc deux branches :

$$\sigma(z) = \begin{cases} 1/(1 + e^{-z}), & z \geq 0, \\ e^z/(1 + e^z), & z < 0. \end{cases}$$

Il est préférable de calculer directement la perte à partir du score,

$$\ell(y, z) = \log(1 + e^z) - yz = \text{softplus}(z) - yz,$$

plutôt que de calculer $\log(\sigma(z))$ après arrondi de la probabilité à 0 ou 1. Cette identité résulte du remplacement de p par $\sigma(z)$ dans la log-perte et constitue la forme usuelle de la perte logistique [7, sec. 7.1].

La fonction `softplus` demande elle-même une évaluation protégée, car e^z déborde pour un score très positif. La forme employée est

$$\text{softplus}(z) = \log(1 + e^z) = \max(z, 0) + \log(1 + e^{-|z|}),$$

dont l'exponentielle porte toujours sur un nombre négatif ou nul.

Contrôle du gradient

Avant tout entraînement, comparer le gradient analytique à une **différence finie centrée** sur un cas test de faible dimension :

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\boldsymbol{\theta} + h\mathbf{e}_j) - J(\boldsymbol{\theta} - h\mathbf{e}_j)}{2h}.$$

Ce test détecte un signe inversé, un facteur $1/m$ oublié ou une transposition mal placée. Sa portée s'arrête à la dérivation et au code du gradient.

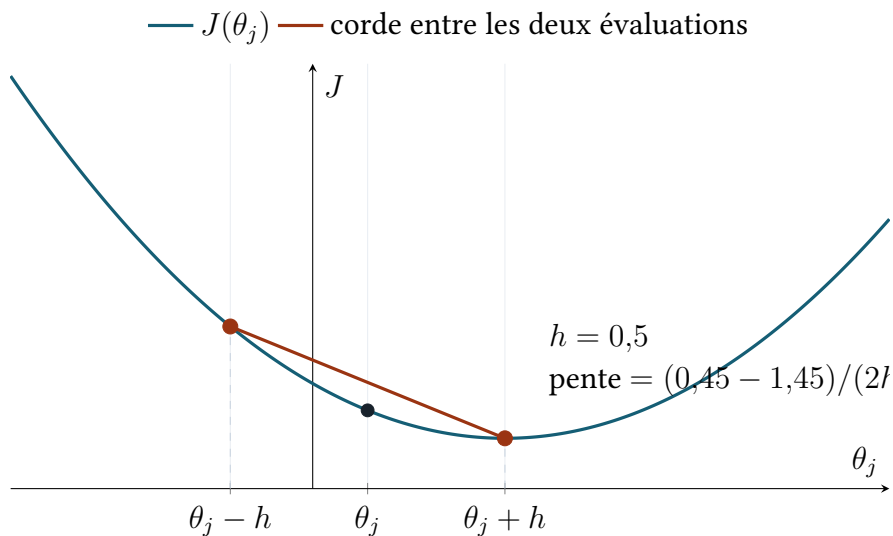


FIGURE 5.5 – Exemple numérique cohérent avec $J(u) = (u - 0,7)^2 + 0,45$, $\theta_j = 0,2$ et $h = 0,5$. Les deux points de la corde appartiennent à la courbe. Sa pente vaut -1 , exactement comme $J'(0,2) = 2(0,2 - 0,7) = -1$ dans cet exemple quadratique.

Sous-procédure — Évaluer les formules sans instabilité numérique

Entrée : un score réel z , éventuellement de grande amplitude. **Sorties** : une probabilité et une perte finies, calculées sans dépassement de capacité.

1. si $z \geq 0$, utiliser $1/(1 + e^{-z})$; sinon utiliser $e^z/(1 + e^z)$, afin que l'exponentielle porte sur un nombre négatif ;
2. calculer la perte depuis z avec une primitive stable de softplus ;
3. tester des scores extrêmes, par exemple $z = -1000, 0, 1000$, et vérifier l'absence de valeur infinie ou indéfinie ;
4. sur un cas de faible dimension, comparer gradient analytique et différence finie avec une tolérance fixée avant l'entraînement complet.

Chapitre 6

Extension multi-classes un-contre-tous

OvR transforme le problème à quatre maisons en quatre questions binaires : « Gryffindor contre les autres », puis Hufflepuff, Ravenclaw et Slytherin. Pour un nouvel élève, les quatre modèles produisent quatre scores ; la maison associée au score maximal devient la prédiction.

Pour chaque classe $k \in \{1, \dots, K\}$, on construit la cible binaire

$$y_{ik} = \mathbf{1}[c_i = k].$$

Cette réduction est la stratégie one-vs-rest : un classifieur binaire par classe, la classe considérée étant positive et toutes les autres négatives [5, sec. 4.1]. On ajuste ensuite indépendamment un paramètre θ_k pour chacun de ces problèmes. On obtient les scores $z_k(\mathbf{x}) = \theta_k^\top \tilde{\mathbf{x}}$ et les sorties binaires $q_k(\mathbf{x}) = \sigma(z_k(\mathbf{x}))$. La règle de décision est

$$\hat{c}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} q_k(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} z_k(\mathbf{x}), \quad (3)$$

car σ est strictement croissante et commune aux K modèles. La décision par maximum des réponses binaires est la règle OvR usuelle [5].

L'apprentissage produit ainsi K vecteurs de paramètres $\theta_1, \dots, \theta_K$. Pour quatre maisons, une même matrice $\mathbf{X}$ est associée successivement à quatre vecteurs cibles $\mathbf{y}_1, \dots, \mathbf{y}_4$. À la prédiction, une observation préparée est multipliée par chacun des quatre vecteurs de paramètres ; les quatre scores obtenus sont comparés, et l'indice du maximum fournit la maison.

classe k	score z_k	sortie $q_k = \sigma(z_k)$	rang
Gryffindor	-1,99	0,12	3
Hufflepuff	-2,31	0,09	4
Ravenclaw	0,90	0,71	1
Slytherin	-1,21	0,23	2

TABLE 6.1 – Décision OvR sur une observation. Ravenclaw maximise à la fois z_k et q_k , la même fonction strictement croissante σ étant appliquée aux quatre scores. La somme des sorties vaut ici 1,15.

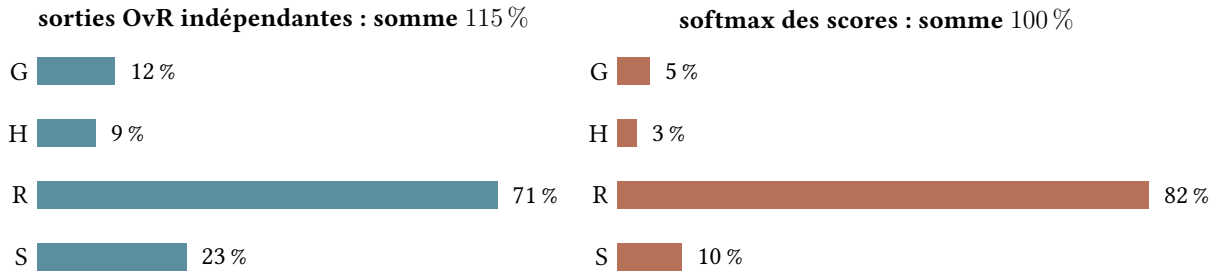


FIGURE 6.1 – Comparaison sur les scores de la table 6.1. OvR applique une sigmoïde séparée à chaque score ; softmax normalise conjointement les exponentielles des scores. Les deux méthodes choisissent ici Ravenclaw, mais leurs valeurs n'ont pas la même signification probabiliste.

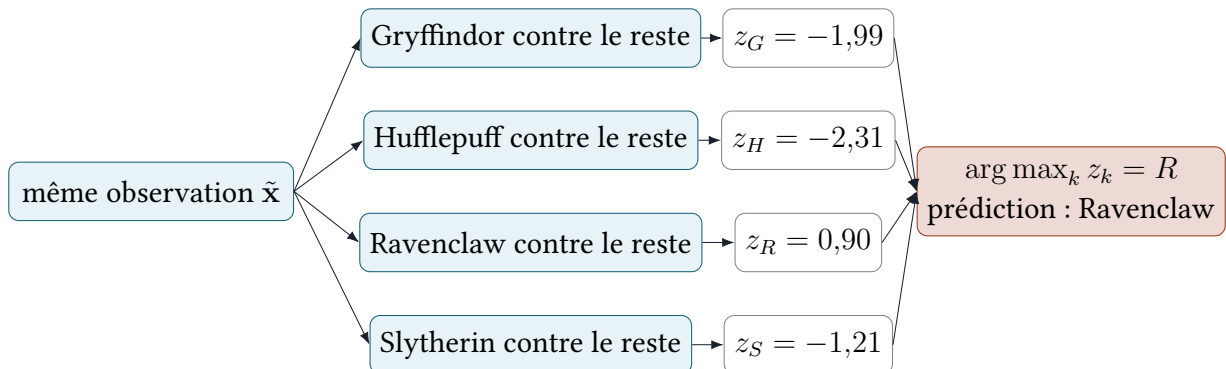


FIGURE 6.2 – Architecture OvR complète pour les quatre maisons. La même observation traverse les quatre modèles binaires, qui produisent quatre scores, ensuite comparés.

Sous-procédure — Ajuster et utiliser un modèle OvR

Entrées d'apprentissage : $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$ et les classes c_i . **État produit :** K vecteurs θ_k associés à K libellés. **Sortie de prédiction :** un libellé de classe.

1. pour la classe k , construire $y_{ik} = 1$ si $c_i = k$, sinon 0;
2. ajuster θ_k avec $\mathbf{X}$ et ce vecteur cible, puis répéter pour les K classes;
3. stocker les paramètres dans un ordre explicite, par exemple G, H, R, S, et conserver cet ordre avec le modèle;
4. pour une nouvelle ligne $\tilde{\mathbf{x}}$, calculer $(z_G, z_H, z_R, z_S) = (\theta_G^\top \tilde{\mathbf{x}}, \dots, \theta_S^\top \tilde{\mathbf{x}})$;
5. prendre la position de la plus grande valeur; par exemple $(-1,99, -2,31, 0,90, -1,21)$ renvoie la troisième position, donc Ravenclaw;
6. vérifier que quatre scores ont été produits et qu'aucun n'est non fini.

Scores, probabilités binaires et probabilités multiclassées

q_k estime la probabilité de la cible binaire « classe k contre le reste » dans le modèle k . Les K modèles étant ajustés séparément, chaque q_k décrit sa propre expérience binaire et la somme $\sum_k q_k$ prend une valeur libre. Ces nombres servent au classement de l'équation (3); leur usage comme vecteur de probabilités multiclassées **calibrées** demande une recalibration préalable [6]. Une régression logistique multinomiale à fonction softmax fournit directement une **loi catégorielle** lorsque cet objectif probabiliste est central [1].

Règle de décision OvR

La décision OvR tient en deux opérations : calculer les quatre scores z_k , puis retenir $\arg \max_k z_k$. La section suivante présente softmax comme un modèle probabiliste multiclassées distinct, avec sa propre estimation.

La fonction **softmax** mentionnée dans la comparaison est

$$\mathbb{P}(C = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{r=1}^K e^{z_r}}.$$

Son dénominateur $\sum_r e^{z_r}$ couple les K classes et impose une somme égale à 1. Dans OvR, le dénominateur $1 + e^{-z_k}$ dépend du seul score z_k . Cette différence algébrique fonde la différence d'interprétation; sur l'exactitude, les deux méthodes se départagent au cas par cas.

Les négatifs d'un problème OvR réunissent les trois autres maisons : ils sont plus nombreux et plus hétérogènes que les positifs. Chaque sous-modèle affiche donc une exactitude binaire élevée par construction. Le jugement porte sur la décision multiclassées finale et sur les performances par maison.

La règle du maximum s'applique dans deux situations remarquables. Quand les quatre scores sont négatifs, elle retient le moins négatif. Quand plusieurs scores sont fortement positifs, elle retient le plus grand. Dans les deux cas elle produit une maison. Une politique de rejet forme une décision supplémentaire, définie et validée à part.

Exemple de rejet : laisser la maison vide si le meilleur score descend sous -2 , ou si l'écart entre les deux meilleurs scores reste sous 0,1. Ces seuils viennent du protocole et introduisent

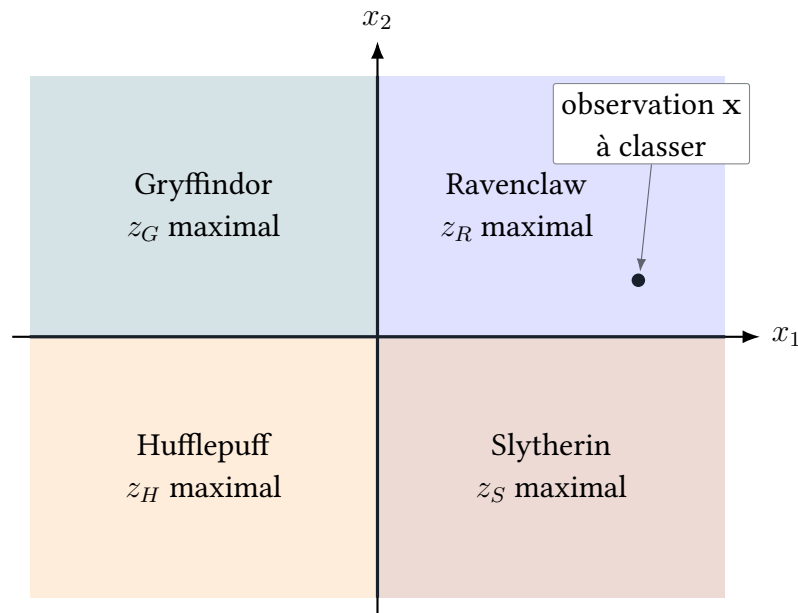


FIGURE 6.3 – Régions de décision OvR dans un espace à deux caractéristiques. Chaque point rejoint la région dont le score domine. Les frontières entre deux régions vérifient $z_k(\mathbf{x}) = z_r(\mathbf{x})$ et restent linéaires tant que les scores le sont.

leur propre arbitrage : davantage de rejets réduit les mauvaises attributions et laisse davantage d'élèves sans réponse. Ils se choisissent sur validation d'après le coût respectif d'un rejet et d'une erreur, puis s'évaluent sur test.

État conservé par le modèle OvR

Avec K classes, l'apprentissage conserve K vecteurs de paramètres et l'ordre qui associe chaque vecteur à son libellé. Une prédiction produit un vecteur de K scores dans ce même ordre, dont l'indice du maximum se convertit en libellé. Cette correspondance appartient au modèle entraîné.

Chapitre 7

Évaluation et expérience reproductible

Un modèle qui annonce 98 % de réussite n’a rien annoncé tant qu’on ignore sur quelles lignes ce nombre a été obtenu. Deux questions restent alors ouvertes : que mesurer, puisqu’un seul pourcentage cache la maison qui résiste, et à quoi comparer le résultat pour savoir s’il vaut mieux qu’une règle triviale.

7.1 Matrice de confusion et mesures

L’exactitude compte la proportion de décisions correctes. La matrice de confusion conserve le détail : ses lignes portent les maisons réelles, ses colonnes les maisons prédites. Chaque case donne un effectif, et la lecture par ligne ou par colonne identifie les classes reconnues et les paires confondues.

Pour un test de taille m_{te} , l’exactitude est

$$\text{accuracy} = \frac{1}{m_{te}} \sum_{i \in \mathcal{I}_{te}} \mathbf{1}[\hat{c}_i = c_i].$$

Cette quantité est l’estimateur empirique de la probabilité de classification correcte sur un échantillon test indépendant [1, chap. 7]. Le français statistique réserve *exactitude* à cette proportion globale et *précision* à $\text{precision} = TP / (TP + FP)$.

L’évaluation reçoit deux vecteurs de même longueur : les classes réelles (c_i) et les classes prédites ($\hat{c}_i$) du test. Chaque paire $(c_i, \hat{c}_i)$ incrémente une case d’une matrice $K \times K$. L’exactitude sort de la diagonale, les mesures par classe des sommes de lignes et de colonnes. L’opération est un comptage.

La *matrice de confusion* M , définie par $M_{ab} = \#\{i : c_i = a, \hat{c}_i = b\}$, localise les maisons confondues. Chaque classe reçoit ensuite son *rappel*, sa *précision* et son *score* F_1 . La *moyenne macro* donne le même poids à chaque maison, la moyenne pondérée tient compte de leurs effectifs. Les deux accompagnent la matrice complète.

Le score F_1 d'une classe est la moyenne harmonique de sa précision P et de son rappel R :

$$F_1 = \frac{2PR}{P + R}.$$

La moyenne harmonique reste proche de la plus petite des deux valeurs : avec $P = 0,80$ et $R = 0,50$, elle donne $F_1 \approx 0,615$ quand la moyenne arithmétique donnerait 0,65.

Supposons ensuite deux classes : la première contient 900 observations et a un F_1 de 0,95 ; la seconde en contient 100 et a un F_1 de 0,40. La moyenne macro vaut $(0,95 + 0,40)/2 = 0,675$. La moyenne pondérée vaut $(900 \times 0,95 + 100 \times 0,40)/1000 = 0,895$. L'écart de 0,22 entre les deux mesure le poids de la classe majoritaire dans le calcul. Rapporter les deux distingue la performance par individu de l'équilibre entre classes.

réelle \ prédite	G	H	R	S	total
G	42	3	2	3	50
H	4	35	6	5	50
R	1	4	43	2	50
S	5	2	3	40	50
total prédit	52	44	54	50	200

TABLE 7.1 – Matrice de confusion construite pour l'exemple. Les décisions correctes occupent la diagonale. L'exactitude vaut $(42 + 35 + 43 + 40)/200 = 0,80$; la ligne Hufflepuff porte un rappel de 0,70, le plus faible des quatre.

Dans cet exemple, le rappel de Hufflepuff vaut $35/50 = 0,70$ et sa précision $35/44 \approx 0,80$. Le rappel répond à « quelle part des vrais Hufflepuff est retrouvée ? », la précision à « parmi les élèves prédits Hufflepuff, quelle part l'est réellement ? ». Le premier divise par la somme de la ligne, le second par la somme de la colonne : deux dénominateurs, deux questions.

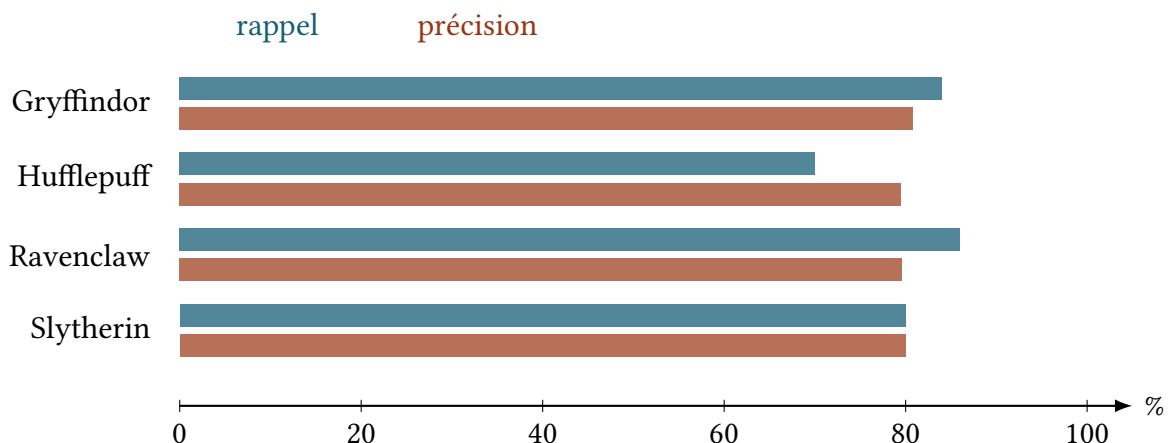


FIGURE 7.1 – Rappel et précision par classe calculés à partir de la table 7.1. Une exactitude globale de 80 % masque notamment le rappel plus faible de Hufflepuff.

Sous-procédure — Construire les mesures de classification

Entrées : m_{te} classes réelles et autant de classes prédites, codées dans le même ordre de K libellés. **Sorties :** une matrice de confusion M , l'exactitude et les mesures par classe.

1. initialiser $M \in \mathbb{N}^{K \times K}$ à zéro ; les lignes représentent le réel et les colonnes la prédiction ;
2. pour chaque paire $(c_i, \widehat{c}_i)$, incrémenter $M_{c_i, \widehat{c}_i}$;
3. vérifier que la somme de toutes les cases vaut m_{te} ;
4. calculer l'exactitude comme somme diagonale divisée par m_{te} ;
5. pour la classe k , calculer rappel = diagonale/somme de la ligne et précision = diagonale/somme de la colonne, puis F_1 ;
6. signaler tout dénominateur nul et rapporter les valeurs par classe.

7.2 Protocole minimal d'évaluation

L'interprétation d'un score demande la manière dont il a été obtenu. Le protocole enregistre les données utilisées, leur préparation, les paramètres d'entraînement et les mesures finales, de sorte que la même expérience puisse être reproduite et contrôlée.

Une expérience exploitable doit conserver :

1. les variables retenues et la politique de valeurs manquantes ;
2. la partition, sa stratification et sa graine ;
3. les statistiques de préparation apprises sur le train ;
4. α , ε , le nombre maximal d'itérations et, le cas échéant, λ ;
5. pour chaque classe, le coût initial, le coût final, le nombre d'itérations et la norme finale du gradient ;
6. sur le test : matrice de confusion, exactitude et mesures par classe.

Deux contrôles situent le résultat. Le premier compare le modèle à une référence majoritaire, qui prédit toujours la maison la plus fréquente. Le second, le **contrôle par permutation**, mélange aléatoirement les étiquettes avant l'entraînement : la performance retombe alors au niveau de cette même règle constante, et tout dépassement signale une fuite de données ou une erreur de protocole.

Exemple : si Gryffindor représente 35 % du test, prédire Gryffindor partout donne déjà 35 % d'exactitude. Un modèle à 38 % se compare donc à cette référence majoritaire autant qu'au hasard uniforme de 25 %. Pour le contrôle par permutation, les maisons sont mélangées entre les lignes et les notes restent en place, ce qui détruit le lien entre caractéristiques et réponse. Le résultat attendu est la performance de la meilleure règle constante ; un modèle qui atteindrait encore 80 % aurait accès à la réponse par les variables ou par la préparation.

Sous-procédure — Exécuter une évaluation contrôlée

Entrées : modèle figé, statistiques de préparation, indices de test et classes réelles. **Sortie :** un compte rendu reproductible, sans modification du modèle.

1. sélectionner les lignes désignées par $\mathcal{I}_{te}$ et leur appliquer les statistiques d'apprentissage déjà conservées ;
2. produire exactement une prédiction par ligne, sans mise à jour des paramètres ;
3. vérifier l'égalité des longueurs entre prédictions et classes réelles ;
4. construire matrice de confusion, exactitude et mesures par classe ;
5. comparer l'exactitude à celle de la classe majoritaire et exécuter séparément le contrôle par permutation sur une expérience d'apprentissage ;
6. enregistrer résultats, paramètres, hyperparamètres, graine et indices.

7.3 Validation croisée : plis et répétitions

Une seule partition avantage ou désavantage un modèle au hasard du découpage. La validation croisée fait tourner le rôle de validation entre plusieurs blocs : chaque observation est évaluée une fois, par un modèle ajusté sur les autres blocs. Répéter l'ensemble du découpage mesure la sensibilité à cette répartition aléatoire.

Un **pli** désigne un bloc d'observations. Dans une **validation croisée** à cinq plis, on découpe les données disponibles pour la sélection du modèle en cinq blocs disjoints $B_1, \dots, B_5$. On entraîne cinq fois le modèle : à chaque fois, quatre blocs servent à l'ajustement et le bloc restant sert à la validation.

	B_1	B_2	B_3	B_4	B_5
validation sur B_1	V	A	A	A	A
validation sur B_2	A	V	A	A	A
validation sur B_3	A	A	V	A	A
validation sur B_4	A	A	A	V	A
validation sur B_5	A	A	A	A	V

A : apprentissage V : validation

FIGURE 7.2 – Validation croisée à cinq plis. Chaque observation sert quatre fois à l'apprentissage et exactement une fois à la validation. Le test final reste à l'écart de l'ensemble de ce mécanisme.

Supposons que les exactitudes de validation des cinq tours soient 0,79, 0,82, 0,77, 0,81 et 0,80. Leur moyenne vaut 0,798, leur étendue 0,05. Cette étendue mesure la dépendance du résultat aux observations placées dans le bloc de validation, et accompagne donc la moyenne dans le compte rendu.

Une *répétition* consiste à refaire tout le découpage en cinq blocs avec une autre permutation aléatoire, puis à recommencer les cinq entraînements. Les observations ne changent pas ; seule leur affectation aux blocs change. Deux répétitions produisent ici dix mesures :

	pli 1	pli 2	pli 3	pli 4	pli 5	moyenne
répétition 1	0,79	0,82	0,77	0,81	0,80	0,798
répétition 2	0,81	0,78	0,80	0,76	0,83	0,796
moyenne des dix validations						0,797

TABLE 7.2 – Exemple de validation croisée répétée. Les ensembles d'apprentissage des dix tours se recouvrent fortement : les dix nombres décrivent la sensibilité au découpage, et la moyenne garde le statut d'estimation unique.

Sous-procédure – Comparer des configurations par validation croisée

Entrées : les données d'apprentissage, une liste de configurations et un nombre de plis q . Un pli est une liste d'indices utilisée une fois pour la validation. **Sortie :** une configuration choisie sans consulter le test.

1. répartir les indices en q blocs disjoints et enregistrer ces blocs ;
2. pour une configuration donnée et un bloc B_r , ajuster sur tous les indices hors de B_r , puis mesurer sur B_r ;
3. répéter pour $r = 1, \dots, q$: chaque ligne sert ainsi une fois à la validation ;
4. calculer la moyenne et la dispersion des q mesures ;
5. recommencer avec exactement les mêmes blocs pour chaque configuration ;
6. choisir selon une règle fixée, puis seulement exécuter le protocole final et le test.

La comparaison de deux modèles A et B emploie exactement les mêmes blocs. Si, sur un pli donné, A obtient 0,81 et B 0,79, la différence vaut +0,02. Répéter ce calcul pli par pli isole l'effet du modèle de la difficulté propre à chaque bloc. Deux découpages distincts ajouteraient la variation des données à celle des modèles.

Incertitude de l'évaluation

Une exactitude test est une estimation, soumise à la variabilité d'échantillonnage. Sur 200 observations, deux prédictions correctes de plus valent un point de pourcentage, à l'intérieur de la marge d'un tirage. La comparaison de modèles conserve donc les mêmes partitions. Les plis et répétitions de la section précédente mesurent la sensibilité pendant la sélection ; le test final reste une estimation avec son intervalle.

Par exemple, A classe correctement 162 élèves sur 200 et B en classe 160. L'écart d'exactitude vaut 0,01, et les décisions appariées le décomposent. Premier cas : A corrige deux erreurs de B et reproduit toutes ses réussites, l'écart tient alors à deux élèves identifiés. Second cas : A et B se trompent chacun sur vingt élèves presque tous différents, et le même total recouvre deux comportements distincts. La matrice de confusion et la liste des désaccords séparent ces deux situations.

Résultats produits par l'évaluation

L'évaluation finale produit le vecteur des classes prédites, la matrice de confusion, l'exactitude globale et les mesures par classe. Les historiques de coût et de norme du gradient décrivent la convergence de l'entraînement, sur l'ensemble d'apprentissage. Les deux séries de nombres répondent à des questions distinctes et figurent toutes deux au compte rendu.

Chapitre 8

Entraînement et prédiction

Deux programmes suffisent à produire le classifieur. Le premier lit les élèves étiquetés et écrit les paramètres ; le second lit ces paramètres et attribue une maison à chaque nouvel élève. Les nombres cités proviennent de `dataset_train.csv`.

8.1 Colonnes du fichier

Dix-neuf colonnes : un identifiant, la maison, quatre champs d'état civil et treize notes. Les notes seules se mesurent sur une échelle numérique, et toutes ne séparent pas les maisons.

Le fichier d'apprentissage contient 1 600 lignes et dix-neuf colonnes.

colonne	contenu et emploi
Index	entier de 0 à 1 599. Recopié dans <code>houses.csv</code> pour apparier chaque prédiction à son élève
Hogwarts House	la réponse c_i . Quatre valeurs, d'effectifs 529, 443, 327 et 301. Source des cibles binaires y_{ik}
First Name, Last Name	identité de l'élève. Écartées
Birthday	date de naissance au format AAAA-MM-JJ, de 1996 à 2001. Écartée d'après la section 8.2
Best Hand	Left pour 790 élèves, Right pour 810. Écartée d'après la section 8.2
treize colonnes de notes	nombres réels. Variables candidates

TABLE 8.1 – Composition du fichier d'apprentissage.

8.2 Best Hand et Birthday

La main dominante et la date de naissance sont disponibles sur toutes les lignes et se codent facilement en nombres. Reste à savoir si elles apprennent quoi que ce soit sur la maison. Trois mesures en décident : la proportion de gauchers maison par maison, l'écart de notes entre gauchers et droitiers, et l'exactitude du modèle avec puis sans ces colonnes.

Ces deux colonnes sont remplies sur les 1 600 lignes et demandent un codage numérique avant d'entrer dans le modèle. Trois mesures décident de leur sort.

La proportion de gauchers varie de 48,1 % chez Ravenclaw à 51,4 % chez Gryffindor, pour 49,4 % sur l'ensemble. Le tableau de contingence 4×2 donne un χ^2 de 0,94 à trois degrés de liberté, quand le seuil à 5 % vaut 7,81.

L'écart entre la note moyenne des gauchers et celle des droitiers reste sous 0,10 écart-type pour les treize cours, avec un maximum de 0,095 pour Defense Against the Dark Arts. L'erreur type d'un tel écart vaut $\sqrt{1/790 + 1/810} = 0,050$ écart-type : les treize valeurs tiennent à moins de deux erreurs types de zéro. Les maisons, elles, se séparent de 1,86 à 2,66 écarts-types sur ces mêmes cours.

Le mois de naissance donne un χ^2 de 33,4 à 33 degrés de liberté. L'année donne 28,1 à 15 degrés de liberté, au-dessus du seuil à 5 % qui vaut 25,0 : une association faible, portée en partie par les 40 élèves nés en 1996 contre environ 320 pour chacune des années suivantes.

Le test décisif porte sur l'exactitude. Best Hand codée ± 1 , puis l'année de naissance standardisée, ajoutées chacune comme onzième variable, laissent l'exactitude hors échantillon à 0,9832 sur cinq graines de découpage, identique aux quatre décimales. Les coefficients de Best Hand s'échelonnent de $-0,20$ à $+0,19$, contre 2,39 pour Astronomy chez Hufflepuff. Les deux colonnes restent hors du modèle.

8.3 Redondance

Deux colonnes qui portent la même information gênent le calcul. Le modèle peut augmenter le coefficient de l'une et diminuer celui de l'autre sans rien changer à ses prédictions : ses coefficients dérivent alors sans jamais se fixer. Une des deux colonnes suffit.

Le jeu de données vérifie

$$\text{Astronomy} = -100 \times \text{Defense Against the Dark Arts}$$

sur les 1 537 lignes où les deux valeurs sont présentes, avec un écart maximal de $4,5 \cdot 10^{-13}$ et une **corrélation** de -1 à six décimales. Le chapitre 4 établit la conséquence : deux colonnes **proportionnelles** rendent $\mathbf{X}^T \mathbf{W} \mathbf{X}$ singulière et laissent la fonction de coût plate le long d'une direction. Les coefficients dérivent alors librement sur cette direction pendant que les scores restent identiques. Astronomy est conservée, Defense Against the Dark Arts sert à l'imputation décrite plus bas.

8.4 Pouvoir discriminant

Une note aide à reconnaître une maison lorsque les quatre maisons y obtiennent des résultats différents. Si les quatre ont la même moyenne, la note ne distingue personne. Le nombre δ_j ci-dessous mesure cet écart entre maisons, dans une unité qui permet de comparer des cours notés sur des échelles très différentes.

Pour la variable j , l'étendue des moyennes par maison rapportée à l'écart-type global vaut

$$\delta_j = \frac{\max_k \bar{r}_{jk} - \min_k \bar{r}_{jk}}{s_j}.$$

Un δ_j de 2 signifie que les maisons extrêmes ont des moyennes séparées par deux écarts-types : leurs distributions se chevauchent peu et la variable sépare. Un δ_j de 0,07 place les quatre moyennes à l'intérieur d'un quatorzième d'écart-type : les quatre histogrammes se superposent.

variable	δ_j	manquantes	décision
Arithmancy	0,068	34	écartée : homogène
Care of Magical Creatures	0,147	40	écartée : homogène
Ancient Runes	1,860	35	retenue
Herbology	1,879	33	retenue
Defense Against the Dark Arts	1,909	31	écartée : redondante
Astronomy	1,911	32	retenue
Muggle Studies	2,038	35	retenue
Potions	2,076	30	retenue
History of Magic	2,221	43	retenue
Transfiguration	2,284	34	retenue
Divination	2,368	39	retenue
Charms	2,466	0	retenue
Flying	2,657	0	retenue

TABLE 8.2 – Les treize notes classées par pouvoir discriminant croissant. Arithmancy et Care of Magical Creatures sont en dessous des autres d'un facteur douze ; la coupure tombe dans un intervalle vide.

Variables retenues

Astronomy, Herbology, Divination, Muggle Studies, Ancient Runes, History of Magic, Transfiguration, Potions, Charms, Flying. Donc $n = 10$, $\mathbf{X} \in \mathbb{R}^{m \times 11}$ et chaque $\theta_k \in \mathbb{R}^{11}$.

8.5 Valeurs manquantes

Une case vide arrête le calcul : le modèle ne sait rien faire d’une note absente. Deux réponses existent, supprimer la ligne ou lui attribuer une valeur. Le choix se joue ici sur le nombre d’élèves que la première ferait perdre.

Onze des treize colonnes de notes comportent des cases vides, entre 30 et 43 chacune, réparties sur 349 lignes distinctes.

Les 1 600 lignes possèdent au moins une des deux valeurs Astronomy ou Defense Against the Dark Arts. La relation de proportionnalité donne donc les 32 valeurs manquantes d’Astronomy à partir de la colonne écartée, au facteur -100 près. Ces 32 cases reçoivent leur valeur exacte, et Astronomy devient complète.

Restent alors 239 lignes portant au moins une case vide parmi les dix variables retenues, soit 14,9 % de l’échantillon : les supprimer ramènerait l’apprentissage à 1 361 élèves. L’imputation médiane du chapitre 2 les conserve toutes, et porte sur les sept colonnes encore incomplètes. Charms, Flying et Astronomy sont pleines.

8.6 Matrice de design

Une seule fonction, appelée par les deux programmes, transforme un fichier en tableau de nombres. Elle applique dans l’ordre : reconstruction d’Astronomy, imputation médiane, standardisation, ajout de la colonne de biais — la sous-procédure du chapitre 2 sur les dix colonnes retenues.

Sous-procédure — Construire $\mathbf{X}$

Entrée : un fichier de données. **Sorties** : $\mathbf{X}$, le vecteur des identifiants Index, et le vecteur des classes lorsque la colonne est remplie.

- lire les lignes dans l’ordre du fichier ; cet ordre est celui de `houses.csv` ;
- extraire les dix colonnes retenues, dans l’ordre fixé par le fichier de poids ;
- convertir les cases en nombres réels ; une case vide devient une valeur manquante ;
- remplir les valeurs manquantes d’Astronomy par -100 fois la note de Defense Against the Dark Arts, puis les valeurs manquantes restantes par la **médiane** d’apprentissage de leur colonne ;
- appliquer $x_{ij} = (r_{ij} - \mu_j)/s_j$ avec les μ_j et s_j d’apprentissage ;
- préfixer chaque ligne d’une coordonnée égale à 1, ce qui donne $\mathbf{X} \in \mathbb{R}^{m \times 11}$.

Sur `dataset_train.csv`, $m = 1\,600$ et $\mathbf{X}$ mesure $1\,600 \times 11$. Sur `dataset_test.csv`, $m = 400$ et $\mathbf{X}$ mesure 400×11 . Le découpage stratifié 80 %/20 % du chapitre 7 partage les 1 600 lignes d’apprentissage en 1 278 et 322.

8.7 Le programme d'entraînement

L'entraînement répète quatre calculs jusqu'à ce que les coefficients cessent de bouger : un score pour chaque élève, la probabilité qui lui correspond, l'écart entre cette probabilité et la bonne réponse, puis une correction des coefficients proportionnelle à cet écart. La même boucle tourne quatre fois, une par maison.

L'entraînement traite les quatre maisons l'une après l'autre. Pour la maison k , il forme la cible $\mathbf{y}_k \in \{0, 1\}^{1600}$ par $y_{ik} = \mathbf{1}[c_i = k]$, part de $\boldsymbol{\theta}_k = \mathbf{0}$, puis répète quatre opérations :

$$\mathbf{z} = \mathbf{X}\boldsymbol{\theta}_k(m), \quad \mathbf{p} = \sigma(\mathbf{z})(m), \quad \mathbf{g} = \frac{1}{m}\mathbf{X}^\top(\mathbf{p} - \mathbf{y}_k) \quad (11), \quad \boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_k - \alpha \mathbf{g}.$$

Le coût J de l'équation (1) sert au critère d'arrêt, le gradient $\mathbf{g}$ est l'équation (2). Le départ à $\boldsymbol{\theta}_k = \mathbf{0}$ donne un résultat reproductible d'une exécution à l'autre, la convexité établie au chapitre 4 garantissant l'unicité de la limite à direction plate près.

Sous-procédure — logreg_train

Entrée : `dataset_train.csv`. **Sortie :** le fichier de poids.

1. calculer médianes, μ_j et s_j sur les lignes d'apprentissage, puis construire $\mathbf{X}$;
2. pour $k = 1, \dots, 4$, former $\mathbf{y}_k$ et itérer les quatre opérations ci-dessus jusqu'à $|J^{(t+1)} - J^{(t)}| / \max(1, |J^{(t)}|) < \varepsilon$ ou épuisement du plafond d'itérations ;
3. relever pour chaque maison le coût initial, le coût final, le nombre d'itérations et $\|\mathbf{g}\|_2$;
4. écrire le fichier de poids avec les statistiques de préparation.

α	ε	itérations	$\ \nabla J\ _2$ final	exactitude hors échantillon
0,5	10^{-6}	1 817	$1,4 \cdot 10^{-3}$	0,981
1	10^{-6}	1 461	$1,0 \cdot 10^{-3}$	0,981
2	10^{-6}	1 150	$7,1 \cdot 10^{-4}$	0,981
1	10^{-7}	5 448	$3,2 \cdot 10^{-4}$	0,981

TABLE 8.3 – Itérations consommées par la maison la plus lente, sur les 1 278 lignes d'apprentissage du découpage stratifié. Diviser ε par dix multiplie les itérations par près de quatre et laisse les 322 décisions du test interne inchangées : la convergence numérique se poursuit après que le classement s'est stabilisé.

Configuration de départ

$\alpha = 1$, $\varepsilon = 10^{-6}$, plafond de 6 000 itérations, $\boldsymbol{\theta}_k = \mathbf{0}$. Cette configuration atteint 0,981 sur un test interne de 322 élèves, et 0,983 en moyenne sur cinq graines de découpage.

8.8 Le fichier de poids

L'entraînement et la prédiction sont deux programmes distincts, lancés à des moments différents et sans mémoire commune. Tout ce que le premier a appris passe au second par un fichier ; ce qui n'y figure pas est perdu.

La prédiction a besoin de trois choses : l'ordre des dix variables, les statistiques qui les transforment, et les quatre vecteurs de paramètres associés à leurs libellés. Les statistiques figurent dans le fichier parce qu'elles sont estimées sur l'apprentissage ; les recalculer sur `dataset_test.csv` placerait les scores dans une autre échelle et ferait chuter l'exactitude.

weights.csv

```
feature,median,mu,sigma
Astronomy,258.1,38.6,520.8
Herbology,3.5,1.2,5.2
[...]
Flying,-2.5,22.0,97.6
class,bias,Astronomy,Herbology,...,Flying
Gryffindor,-3.54,0.50,-1.81,...,1.61
Hufflepuff,-2.36,2.48,1.63,...,-0.48
Ravenclaw,-2.56,-1.37,0.74,...,-0.17
Slytherin,-3.77,-2.02,-0.70,...,-0.59
```

Ces valeurs sont celles produites par le programme de la section 8.10 sur les 1 600 lignes du fichier d'apprentissage. L'écart-type d'Astronomy vaut 521 et celui d'Herbology 5,2 : un facteur cent entre deux colonnes que le même α doit faire progresser, ce que la standardisation ramène à 1.

L'ordre des colonnes et l'ordre des classes appartiennent au modèle. Un vecteur θ_k s'applique aux variables dans l'ordre où il a été ajusté, et l'indice renvoyé par $\arg \max$ se lit dans la liste des libellés enregistrée à côté de lui. Les deux ordres figurent explicitement dans le fichier.

8.9 Primitives à écrire

Le sujet écarte les fonctions toutes faites de statistique descriptive. Cinq suffisent, toutes en une passe ou deux sur un tableau. `median` fournit les valeurs d'imputation, moyenne et `ecart_type` les μ_j et s_j de la standardisation. `sigmoide` et `softplus` tirent la probabilité et la perte du score, sous les formes à deux branches du chapitre 5 : l'exponentielle y porte toujours sur $-|z|$. Écrites naïvement, elles évaluent e^{-z} et e^z , qui débordent à $|z| \approx 710$ en double précision ; les formes ci-dessous rendent 0 et $|z|$.

primitives

```
fonction moyenne(valeurs) :  
    somme ← 0  
    pour valeur dans valeurs :  
        somme ← somme + valeur  
    retourner somme / longueur(valeurs)  
  
fonction ecart_type(valeurs) :  
    centre ← moyenne(valeurs) ; somme ← 0  
    pour valeur dans valeurs :  
        somme ← somme + (valeur - centre)2  
    retourner racine(somme / longueur(valeurs))  
  
fonction mediane(valeurs) :  
    trieés ← trier(valeurs)  
    nb ← longueur(trieés)  
    si nb impair :  
        retourner trieés[nb // 2]  
    retourner (trieés[nb // 2 - 1] + trieés[nb // 2]) / 2  
  
fonction sigmoïde(score) :  
    si score ≥ 0 :  
        retourner 1 / (1 + exp(-score))  
    retourner exp(score) / (1 + exp(score))  
  
fonction softplus(score) :  
    retourner max(score, 0) + log(1 + exp(-abs(score)))
```

8.10 Pseudo-code de logreg_train

X compte une ligne par élève et onze colonnes : le biais, puis les dix notes dans l'ordre de MATIERES.

8.10.1 Constantes

L'ordre des dix matières vient de la section 8.4, celui des quatre maisons est arbitraire mais définitif : tous deux partent dans `weights.csv`, la prédiction n'ayant aucun autre moyen de rattacher un vecteur de coefficients à une maison. `MAX_ITER` borne le temps de calcul si le critère d'arrêt tarde ; les trois constantes valent ce que fixe la table 8.3.

constantes et lecture

```
MATIERES ← [Astronomy, Herbology, Divination, Muggle Studies,
            Ancient Runes, History of Magic, Transfiguration,
            Potions, Charms, Flying]
MAISONS ← [Gryffindor, Hufflepuff, Ravenclaw, Slytherin]
DEFENSE ← "Defense Against the Dark Arts"
ALPHA ← 1.0 ; EPS ← 1e-6 ; MAX_ITER ← 6000

eleves ← lire_csv(argv[1])
```

8.10.2 Imputation et statistiques de colonne

La première boucle est propre à ce jeu de données : la proportionnalité de la section 8.3 reconstitue les 32 valeurs manquantes d'Astronomy depuis la colonne écartée. La seconde suit l'ordre du chapitre 2, imputer puis mesurer : μ et σ portent sur la colonne complétée, non sur les seules valeurs présentes. Ces trois quantités par matière sont estimées ici et nulle part ailleurs, faute de quoi le test recevrait une préparation différente de l'apprentissage. Un écart-type nul arrête le programme : la colonne est constante, elle ne se divise pas et ne distingue personne.

imputation et statistiques de colonne

```
pour chaque eleve :
    si eleve[Astronomy] est vide et eleve[DEFENSE] est rempli :
        eleve[Astronomy] ← -100 * eleve[DEFENSE]

pour matiere dans MATIERES :
    notes ← [eleve[matiere] pour chaque eleve, si rempli]
    med[matiere] ← mediane(notes)

    pour chaque eleve :
        si eleve[matiere] est vide :
            eleve[matiere] ← med[matiere]

    notes ← [eleve[matiere] pour chaque eleve]
    mu[matiere] ← moyenne(notes)
    sd[matiere] ← ecart_type(notes)
    si sd[matiere] = 0 :
        erreur("colonne constante", matiere)
```

8.10.3 Construction de X

La première colonne, constante, porte θ_0 : sans elle, la frontière serait contrainte de passer par l'origine, comme le montre la figure 2.2. Les dix autres reçoivent les notes standardisées. Passé cette boucle, aucune note brute n'intervient plus.

matrice de design

```
X ← []  
pour chaque eleve :  
  x ← [1.0]  
  pour matiere dans MATIERES :  
    ajouter (eleve[matiere] - mu[matiere]) / sd[matiere] a x  
  ajouter x a X
```

8.10.4 Descente de gradient, maison par maison

cible vaut 1 pour les élèves de la maison traitée et 0 pour les trois autres maisons réunies : c'est $y_{ik} = 1[c_i = k]$, la construction un-contre-tous du chapitre 6. Le départ à $\theta_k = \mathbf{0}$ ne détermine pas le point d'arrivée, la convexité du chapitre 4 s'en chargeant ; il rend l'exécution reproductible.

descente de gradient

```
pour maison dans MAISONS :  
  cible ← [1 si eleve[House] = maison sinon 0, par eleve]  
  poids ← [0.0] * 11  
  cout_prec ← +infini  
  
  repeter MAX_ITER fois :  
    cout ← 0 ; pente ← [0.0] * 11  
  
    pour chaque eleve, sa ligne x et sa cible attendue :  
      score ← 0  
      pour chaque colonne :  
        score ← score + poids[colonne] * x[colonne]  
  
      cout ← cout + softplus(score) - attendue * score  
      ecart ← sigmoide(score) - attendue  
      pour chaque colonne :  
        pente[colonne] ← pente[colonne] + ecart * x[colonne]  
  
    cout ← cout / m  
    pour chaque colonne :  
      pente[colonne] ← pente[colonne] / m  
  
    si abs(cout_prec - cout) / max(1, abs(cout)) < EPS :  
      sortir de la boucle  
    cout_prec ← cout  
  
    pour chaque colonne :  
      poids[colonne] ← poids[colonne] - ALPHA * pente[colonne]  
  
  theta[maison] ← poids
```

Les deux boucles sur les colonnes sont les deux produits matriciels du cycle représenté figure 5.4 : la première accumule $z_i = \theta_k^\top \tilde{\mathbf{x}}_i$, la seconde $\sum_i (p_i - y_i) x_{ij}$, soit $\mathbf{X}\theta_k$ et $\mathbf{X}^\top(\mathbf{p} - \mathbf{y}_k)$. Une bibliothèque de calcul matriciel les remplace par $z = \mathbf{X} @ \text{poids}$ et $\text{pente} = \mathbf{X.T} @ (\mathbf{p} - \text{cible}) / m$. Développées, elles laissent voir ce que la notation compacte : chaque élève pèse sur le gradient à proportion de son erreur et de ses propres notes.

L'accumulation de cout est l'équation (1) sous la forme $\ell(y, z) = \text{softplus}(z) - yz$, qui court-circuite la probabilité. Au-delà de $z = 37$, $\sigma(z)$ s'arrondit à 1 en double précision : la forme directe donnerait $\log(1 - p) = -\infty$ pour un élève d'une autre maison que le modèle place ici avec assurance, et le coût cesserait d'être exploitable au moment où il signale une erreur grave. ecart est le résidu $p_i - y_i$: $-0,9$ pour un Hufflepuff auquel le modèle n'accorde encore que 0,1, $-0,01$ pour un Hufflepuff déjà reconnu à 0,99. La division de pente par m achève l'équation (2).

L'ordre des trois dernières instructions n'est pas libre. Le critère d'arrêt compare deux coûts calculés avant toute modification des poids ; il porterait sinon sur deux vecteurs différents. Et `pente` est complété avant la première affectation à `poids`, si bien que les onze composantes descendent depuis le même ancien vecteur : c'est la mise à jour simultanée du chapitre 5.

L'appel final enregistre l'ordre des matières, les trois statistiques par colonne, l'ordre des maisons et les quatre vecteurs de coefficients, au format de la section 8.8.

écriture du modèle

```
ecrire_poids("weights.csv", MATIERES, med, mu, sd, MAISONS, theta)
```

Exécuté sur les 1 600 lignes, ce programme consomme 436 itérations pour Hufflepuff, 460 pour Ravenclaw, 836 pour Slytherin et 1 069 pour Gryffindor, sous le plafond de 6 000. Les coûts finaux vont de 0,044 à 0,069, et le modèle reclasse correctement 0,982 de son propre ensemble d'apprentissage.

8.11 Le programme de prédiction

Le second programme n'apprend rien. Il relit les nombres écrits par le premier, leur applique la même préparation qu'à l'entraînement, calcule quatre scores par élève et retient la maison du plus grand. Aucune moyenne, aucune médiane et aucun écart-type n'y est recalculé.

La prédiction lit les paramètres, construit X avec les statistiques du fichier de poids, calcule quatre scores par ligne et retient le plus grand, selon l'équation (3). C'est l'architecture de la figure 6.2, appliquée aux 400 lignes du fichier de test. La sigmoïde est croissante et commune aux quatre modèles : comparer les z_k donne le même classement que comparer les q_k .

La remarque s'étend à [softmax](#), dont le dénominateur est commun aux quatre classes : la section 9.7 en donne l'argument. Les trois quantités z_k , $\sigma(z_k)$ et $\text{softmax}(z)_k$ désignent le même argmax , pour tout jeu de scores. Sur les 1 600 lignes du fichier d'apprentissage, les trois règles produisent la même maison, avec zéro ex æquo.

Deux mesures situent la difficulté du problème. L'écart entre le meilleur score et le second vaut 2,05 au minimum et 8,69 en médiane. Le maximum des quatre scores est positif sur chaque ligne, et une seule classe y dépasse 2. Les quatre maisons occupent donc des régions franchement séparées, ce qui éclaire le résultat suivant : une régression softmax multinomiale, ajustée conjointement sur les mêmes découpages, produit les 322 mêmes prédictions que OvR à chaque graine, pour une exactitude moyenne identique de 0,983. Sur ce jeu de données, le choix entre les deux modèles porte sur l'interprétation des sorties, traitée au chapitre 6.

Sous-procédure — logreg_predict

Entrées : `dataset_test.csv` et le fichier de poids. **Sortie :** `houses.csv`.

1. lire l'ordre des variables, les statistiques de préparation, l'ordre des classes et les quatre θ_k ;
2. construire $\mathbf{X} \in \mathbb{R}^{400 \times 11}$ avec ces statistiques;
3. calculer $\mathbf{Z} = \mathbf{X}\Theta^\top \in \mathbb{R}^{400 \times 4}$, où Θ empile les quatre vecteurs de paramètres;
4. pour chaque ligne, prendre l'indice du maximum et le convertir en libellé;
5. écrire l'en-tête puis 400 lignes, dans l'ordre du fichier d'entrée.

Les valeurs d'imputation `med` viennent du fichier de poids, donc des lignes d'apprentissage. Prendre la médiane des 400 lignes de test à la place ferait dépendre la maison d'un élève des autres élèves du même envoi, et changerait ses notes complétées d'un fichier de test à l'autre.

relecture du modèle et complétion des notes

```
DEFENSE ← "Defense Against the Dark Arts"
MATIERES, med, mu, sd, MAISONS, theta ← lire_poids(argv[2])
eleves ← lire_csv(argv[1])

pour chaque eleve :
  si eleve[Astronomy] est vide et eleve[DEFENSE] est rempli :
    eleve[Astronomy] ← -100 * eleve[DEFENSE]

  pour matiere dans MATIERES :
    si eleve[matiere] est vide :
      eleve[matiere] ← med[matiere]
```

Chaque élève est standardisé avec les `mu` et `sd` du fichier, précédé de la coordonnée 1 du biais, puis multiplié par les quatre vecteurs de coefficients. La boucle sur `k` évalue l'arg max de l'équation (3) en une passe, sans conserver les quatre scores. L'initialisation de `score_max` à $-\infty$ couvre le cas où les quatre sont négatifs : la règle retient alors le moins négatif, et une maison sort dans tous les cas.

décision élève par élève

```
ouvrir sortie "houses.csv"
ecrire "Index,Hogwarts House"

pour chaque eleve :
  x ← [1.0]
  pour matiere dans MATIERES :
    ajouter (eleve[matiere] - mu[matiere]) / sd[matiere] a x

  meilleure ← indefinie ; score_max ← -infini
  pour maison dans MAISONS :
    score ← 0
    pour chaque colonne :
      score ← score + theta[maison][colonne] * x[colonne]
    si score > score_max :
      score_max ← score ; meilleure ← maison

  ecrire eleve[Index] + "," + meilleure
```

theta est indexé par les libellés relus en tête de programme. Un fichier de poids écrit sans cette liste, ou dans un autre ordre que celui de l’ajustement, donnerait des scores justes et des maisons fausses.

houses.csv

```
Index,Hogwarts House
0,Gryffindor
1,Hufflepuff
2,Ravenclaw
3,Hufflepuff
[...]
399,Gryffindor
```

Les lignes de `dataset_test.csv` dont plusieurs notes manquent reçoivent une maison comme les autres : leurs cases vides ont été remplies par les médianes d’apprentissage à la lecture. Le fichier produit compte 401 lignes, l’en-tête et une par élève.

8.12 Contrôles

Un programme qui rend un résultat plausible peut être faux. Chacun des contrôles ci-dessous vise une erreur précise et a une valeur attendue connue d’avance.

La colonne `Hogwarts House` de `dataset_test.csv` est vide. L’exactitude se mesure donc sur un ensemble de test découpé dans `dataset_train.csv`, selon le protocole du chapitre 7. Le sujet fixe le seuil d’acceptation à 0,98, mesuré par l’évaluateur sur `dataset_test.csv`.

contrôle	valeur attendue	ce qu'il vérifie
gradient analytique contre différence finie centrée	écart relatif $< 10^{-6}$	l'équation (2) et son code
sigmoïde et perte évaluées en $z = \pm 1000$	valeurs finies	les formes stables du chapitre 5
exactitude d'une prédiction constante sur Hufflepuff	0,331	le niveau que le modèle doit dépasser
exactitude après permutation aléatoire des étiquettes	$\approx 0,34$	l'absence de fuite de données
exactitude sur le test interne	$\geq 0,98$	le protocole du chapitre 7
lignes de houses.csv	401	le format attendu par l'évaluateur

TABLE 8.4 – Valeurs de référence mesurées sur ce jeu de données. La colonne de droite indique l'erreur que chaque contrôle intercepte.

Le contrôle par permutation mélange les maisons entre les lignes avant l'entraînement. Le chapitre 7 situe le résultat attendu à 0,25 pour quatre classes équilibrées. Les effectifs valent ici 529, 443, 327 et 301 : privé de tout lien entre notes et maison, le modèle converge vers la prédiction constante sur la classe majoritaire et atteint 0,34. La référence d'un contrôle par permutation est la performance de la meilleure règle constante.

Sortie de l'entraînement

Quatre vecteurs de onze coefficients, dix médianes, dix moyennes, dix écarts-types, la liste ordonnée des variables et celle des maisons. Ces 74 nombres et ces deux listes forment l'intégralité de ce que lit la prédiction.

Chapitre 9

Notations et lecture des formules

Chaque formule met en jeu des objets d'une nature précise et une opération qui les relie. Les tableaux de ce chapitre donnent pour chaque symbole sa nature, sa lecture orale et son rôle dans le modèle.

Ce chapitre fixe le sens des symboles et la manière de les nommer à l'oral. La nature d'une notation gouverne les opérations qu'elle admet : un scalaire, un vecteur, une matrice, une fonction et un ensemble se manipulent selon des règles propres. La typographie du document la signale — vecteurs en gras, ensembles en lettres calligraphiques, paramètres grecs en gras lorsqu'ils forment un vecteur.

Le document place l'indice d'observation en bas, selon la convention statistique : $\mathbf{x}_i$ est le vecteur de l'élève i , x_{ij} sa caractéristique j et c_i sa classe. Certains cours de machine learning écrivent $\mathbf{x}^{(i)}$ pour séparer le numéro d'exemple d'une puissance ; l'écriture x_{ij} retenue ici s'accorde directement avec l'usage matriciel et se dicte plus simplement. Les crochets en exposant, comme $\theta^{[t]}$, portent le numéro d'itération.

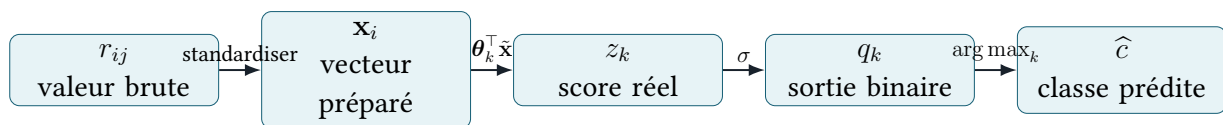


FIGURE 9.1 – Nature des objets successifs et transformation qui mène de l'un à l'autre : une valeur brute, un vecteur préparé, un score réel, une sortie logistique, une classe.

9.1 Indices, effectifs et ensembles

L'écriture $i \in \mathcal{I}_{\text{tr}}$ se lit « i appartient à l'ensemble des indices d'apprentissage ». L'écriture $k \in \{1, \dots, K\}$ se lit « k varie de un à K ».

symbole	lecture orale	signification exacte
i	« indice i » lorsqu'il est attaché à un symbole	indice d'une observation : ici, un élève. Il varie de 1 à m .
j	« indice j » lorsqu'il est attaché à un symbole	indice d'une caractéristique : ici, un cours ou une note. Il varie de 1 à n .
k	« indice k » lorsqu'il est attaché à un symbole	indice d'une classe : ici, une maison. Il varie de 1 à K .
t	« itération t »	indice d'une itération de l'algorithme d'optimisation.
m	« m »	nombre total d'observations dans l'ensemble considéré.
n	« n »	nombre de caractéristiques utilisées par le modèle, avant ajout du biais.
K	« K »	nombre de classes ; dans ce problème, $K = 4$.
$\mathcal{C}$	« l'ensemble des classes C »	ensemble des classes possibles, et non une classe particulière. On précise « C calligraphique » seulement si l'on doit dicter la graphie.
$\mathbb{R}$	« R » ou « l'ensemble des réels »	ensemble dans lequel prennent leurs valeurs les scores, paramètres et caractéristiques numériques.
$\mathcal{I}_{\text{tr}}$	« I indice apprentissage »	ensemble des indices appartenant à l'échantillon d'apprentissage.
$\mathcal{I}_{\text{te}}$	« I indice test »	ensemble des indices appartenant à l'échantillon de test.

TABLE 9.1 – Indices, tailles et ensembles. Un même indice conserve le même rôle dans tout le document.

symbole	lecture orale	signification exacte
r_{ij}	« r indice i j »	valeur brute de la caractéristique j pour l'élève i , avant standardisation. Le premier indice repère la ligne, le second la variable.
μ_j	« mu indice j »	moyenne de la caractéristique j , estimée uniquement sur l'apprentissage.
s_j	« s indice j »	écart-type de la caractéristique j , estimé uniquement sur l'apprentissage.
x_{ij}	« x indice i j »	caractéristique j standardisée de l'observation i ; c'est un scalaire.
$\mathbf{x}_i$	« x indice i »	vecteur colonne des n caractéristiques standardisées de l'observation i ; $\mathbf{x}_i \in \mathbb{R}^n$. Le caractère gras est généralement implicite à l'oral.
$\tilde{\mathbf{x}}_i$	« x tilde indice i » ou « x i augmenté »	vecteur $\mathbf{x}_i$ précédé d'une coordonnée égale à 1 pour intégrer le biais; $\tilde{\mathbf{x}}_i \in \mathbb{R}^{n+1}$.
$\mathbf{X}$	« la matrice X » ou « la matrice de design »	matrice qui empile les vecteurs augmentés en lignes; $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$.
X_i	« la variable aléatoire X indice i »	vecteur aléatoire représentant les caractéristiques de l'observation i avant leur réalisation. Il se distingue de la valeur observée $\mathbf{x}_i$ et de la matrice $\mathbf{X}$.

TABLE 9.2 – Les minuscules désignent une observation ou une composante; la majuscule grasse $\mathbf{X}$ réunit toutes les observations sous forme matricielle.

9.2 Une observation et ses caractéristiques

La formule

$$x_{ij} = \frac{r_{ij} - \mu_j}{s_j}$$

se lit : « x indice i j égale r indice i j moins μ indice j , sur s indice j ». Elle exprime la valeur de la note en nombre d'écart-types au-dessus ou au-dessous de la moyenne d'apprentissage.

9.3 Classes observées, cibles binaires et prédictions

symbole	lecture orale	signification exacte
C_i	« la variable aléatoire C indice i »	maison de l'élève i avant observation ; C_i prend ses valeurs dans $\mathcal{C}$.
c_i	« c indice i »	maison effectivement observée pour l'élève i ; $c_i \in \mathcal{C}$. La minuscule désigne ici une réalisation de C_i .
y_{ik}	« y indice i k »	cible binaire du classifieur k pour l'observation i : 1 si $c_i = k$, sinon 0.
$\mathbf{y}_k$	« le vecteur y indice k »	vecteur colonne réunissant les m cibles du problème « classe k contre le reste » ; $\mathbf{y}_k \in \{0, 1\}^m$.
$\hat{y}_i$	« y chapeau indice i »	classe binaire prédite pour l'observation i , égale à 0 ou 1. Le chapeau marque une valeur estimée.
$\hat{c}(\mathbf{x})$	« c chapeau évalué en $\mathbf{x}$ » ou « la classe prédite en $\mathbf{x}$ »	règle de classification multiclassées évaluée pour les caractéristiques $\mathbf{x}$.
$1[A]$	« l'indicatrice de l'événement A »	vaut 1 lorsque la proposition A est vraie et 0 sinon.

TABLE 9.3 – La classe observée c_i prend ses valeurs dans $\mathcal{C}$; la cible binaire y_{ik} , construite pour le sous-problème OvR numéro k , vaut 0 ou 1.

L'écriture $y_{ik} = 1[c_i = k]$ se lit : « y indice i k est l'indicatrice de l'événement c indice i égale k » ; en contexte, on dira plus naturellement : « la cible de l'observation i pour la classe k vaut un si sa classe est k ».

9.4 Paramètres, score et probabilité binaire

Le produit $\boldsymbol{\theta}^\top \tilde{\mathbf{x}}$ se lit « θ transposé $\tilde{x}$ », ou « le produit scalaire de θ et $\tilde{x}$ ». Le symbole $\top$ indique la transposition : il transforme ici le vecteur colonne $\boldsymbol{\theta}$ en vecteur ligne afin que le produit scalaire soit défini. Développé, ce produit vaut

$$\boldsymbol{\theta}^\top \tilde{\mathbf{x}} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n.$$

symbole	lecture orale	signification exacte
θ_j	« thêta indice j »	coefficient scalaire associé à la caractéristique j . Son signe indique le sens de son effet sur les log-cotes, toutes choses égales par ailleurs.
θ_0	« thêta zéro »	biais ou constante du modèle ; il multiplie la coordonnée ajoutée $x_0 = 1$.
$\boldsymbol{\theta}$	« thêta » ou « le vecteur des paramètres thêta »	vecteur colonne de tous les paramètres $(\theta_0, \dots, \theta_n)^\top \in \mathbb{R}^{n+1}$. On ne dit « vecteur » que lorsque sa nature doit être soulignée.
$\boldsymbol{\theta}_k$	« thêta indice k »	vecteur des paramètres du classifieur binaire « classe k contre le reste ».
$z_{\boldsymbol{\theta}}(\mathbf{x})$	« le score z , paramétré par thêta, évalué en $\mathbf{x}$ » ; souvent « le score en $\mathbf{x}$ »	score linéaire réel $\boldsymbol{\theta}^\top \tilde{\mathbf{x}}$, à valeurs dans $\mathbb{R}$ tout entier.
σ	« sigma »	fonction logistique $z \mapsto (1 + e^{-z})^{-1}$, qui transforme un score réel en nombre strictement compris entre 0 et 1.
$p_{\boldsymbol{\theta}}(\mathbf{x})$	« p paramétré par thêta, évalué en $\mathbf{x}$ » ; souvent « la probabilité prédite en $\mathbf{x}$ »	probabilité conditionnelle modélisée de la classe binaire positive : $\mathbb{P}(Y = 1 \mid X = \mathbf{x})$.
$q_k(\mathbf{x})$	« q indice k évalué en $\mathbf{x}$ »	sortie du classifieur binaire k , égale à $\sigma(\boldsymbol{\theta}_k^\top \tilde{\mathbf{x}})$. La somme des K valeurs OvR prend une valeur libre.
Θ	« thêta majuscule » ou « la matrice des paramètres »	matrice regroupant les K vecteurs de paramètres, un par classifieur OvR.

TABLE 9.4 – Le modèle produit successivement trois objets : le score z dans $\mathbb{R}$, la probabilité binaire p dans $]0, 1[$, la décision $\hat{y}$ dans $\{0, 1\}$.

9.5 Vraisemblance, perte et optimisation

symbole	lecture orale	signification exacte
$L(\theta)$	« la vraisemblance évaluée en θ »	vraisemblance : probabilité des étiquettes observées, considérée comme fonction des paramètres.
$\log L(\theta)$	« la log-vraisemblance en θ »	logarithme de la vraisemblance ; il remplace les produits par des sommes sans changer le maximiseur.
$\ell_i(\theta)$	« ℓ indice i évaluée en θ » ou « la perte de l'observation i »	opposé de la contribution de l'observation i à la log-vraisemblance.
$J(\theta)$	« J évaluée en θ » ou « le coût en θ »	fonction objectif : moyenne des pertes sur l'échantillon d'apprentissage.
$\nabla J(\theta)$	« le gradient de J en θ »	vecteur des $n + 1$ dérivées partielles de J ; il pointe dans la direction locale de plus forte augmentation. « Nabla J » sert à dicter la formule.
$\nabla^2 J(\theta)$	« la Hessienne de J en θ »	matrice $(n + 1) \times (n + 1)$ des dérivées secondes ; elle décrit la courbure locale de la fonction objectif. « Nabla carré J » nomme la notation, « Hessienne » nomme l'objet.
W	« la matrice W »	matrice diagonale dont le terme i vaut $p_i(1 - p_i)$; elle intervient dans la Hessienne.
α	« alpha »	taux d'apprentissage, c'est-à-dire longueur multiplicative du pas de descente de gradient.
λ	« lambda »	intensité de la pénalisation quadratique ; $\lambda = 0$ signifie absence de régularisation.
ε	« epsilon »	tolérance numérique utilisée dans le critère d'arrêt.

TABLE 9.5 – Objets employés pour estimer les paramètres. La vraisemblance est maximisée ; son opposé logarithmique J est minimisé.

La formule

$$\nabla J(\theta) = \frac{1}{m} \mathbf{X}^\top (\sigma(\mathbf{X}\theta) - \mathbf{y})$$

se lit : « le gradient de J en θ vaut un sur m , X transposé, multiplié par le vecteur sigma de $X\theta$ moins y ». Le vecteur entre parenthèses contient les écarts entre probabilités prédites et étiquettes observées.

La mise à jour

$$\theta^{[t+1]} = \theta^{[t]} - \alpha \nabla J(\theta^{[t]})$$

se lit : « à l'itération t plus un, θ vaut sa valeur à l'itération t moins α fois le gradient évalué en cette valeur ». Les crochets $[t]$ repèrent le numéro d'itération.

9.6 Probabilités et opérateurs

symbole	lecture orale	signification exacte
$\mathbb{P}(A)$	« probabilité de l'événement A »	probabilité de l'événement A .
$\mathbb{P}(A \mid B)$	« probabilité de A conditionnellement à B » ; couramment « A sachant B »	probabilité conditionnelle de A lorsque l'information B est connue.
$Y \mid X = \mathbf{x}$	« la loi de Y conditionnellement à X égal $\mathbf{x}$ »	loi conditionnelle de la réponse pour une observation dont les caractéristiques valent $\mathbf{x}$.
$\sum_{i=1}^m$	« somme sur i , de un à m »	addition de l'expression située à droite pour toutes les observations.
$\prod_{i=1}^m$	« produit sur i , de un à m »	multiplication de l'expression située à droite pour toutes les observations.
$\arg \max_k a_k$	« l'argument du maximum sur k de a_k »	indice k pour lequel a_k atteint son maximum ; le résultat est cet indice, quand $\max_k a_k$ désigne la valeur atteinte. Dans l'usage, « $\arg \max$ » est également courant.
$\ v\ _2$	« norme euclidienne de v » ou « norme deux de v »	longueur euclidienne du vecteur v .
$\text{diag}(a_i)$	« la matrice diagonale de termes a_i à l'indice i »	matrice dont la diagonale contient les a_i et dont les autres termes sont nuls.
$\partial J / \partial \theta_j$	« dérivée partielle de J par rapport à θ_j à l'indice j »	variation locale de J lorsque seul θ_j varie, les autres paramètres étant maintenus fixes.

TABLE 9.6 – Opérateurs qui apparaissent dans les dérivations et les règles de décision.

Enfin,

$$\hat{c}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} \theta_k^\top \tilde{\mathbf{x}}$$

se lit : « la classe prédite en $\mathbf{x}$ est l'argument qui maximise, sur les classes k , le score $\theta_k^\top \tilde{\mathbf{x}}$ ». Cette formule renvoie le nom de la classe gagnante après conversion de l'indice vers le libellé correspondant.

9.7 Loïs et fonctions

Loi de Bernoulli

Loi d'une variable aléatoire à deux issues, codées 1 et 0, de paramètre $p \in [0, 1] : \mathbb{P}(Y = 1) = p$

et $\mathbb{P}(Y = 0) = 1 - p$. Les deux cas se réunissent en $\mathbb{P}(Y = y) = p^y(1-p)^{1-y}$, puisque l'exposant annule l'un des deux facteurs. C'est la loi d'un lancer de pièce truquée. Le chapitre 4 l'utilise conditionnellement : chaque élève i reçoit son propre paramètre p_i , calculé depuis ses notes.

Cotes et log-cotes

Les cotes d'un événement de probabilité p valent $p/(1-p)$: le rapport entre sa probabilité et celle de son contraire. Une probabilité de 0,75 donne des cotes de 3, soit « trois contre un ». Les cotes couvrent $[0, +\infty[$ quand p couvre $[0, 1]$. Leur logarithme, ou *log-cotes*, couvre $\mathbb{R}$ tout entier, ce qui le rend compatible avec une fonction affine des caractéristiques. Le terme anglais est *odds* et *log-odds*.

Logit

Nom de la fonction $p \mapsto \log(p/(1-p))$, qui envoie $]0, 1[$ sur $\mathbb{R}$. Le modèle logistique pose que le logit de la probabilité conditionnelle est une fonction affine des caractéristiques. La sigmoïde est sa réciproque.

Fonction logistique, ou sigmoïde

$\sigma(z) = 1/(1 + e^{-z})$. Elle envoie $\mathbb{R}$ sur $]0, 1[$, croît strictement, vaut $1/2$ en 0 et vérifie $\sigma(-z) = 1 - \sigma(z)$ ainsi que $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Sa forme en S lui donne son nom. Sa stricte croissance a une conséquence employée au chapitre 6 : comparer des $\sigma(z_k)$ revient à comparer les z_k .

Softplus

$\text{softplus}(z) = \log(1 + e^z)$, primitive de la sigmoïde. Elle approche 0 pour z très négatif et z pour z très positif, d'où son nom : une version lissée de $\max(z, 0)$. Le chapitre 5 l'emploie pour calculer la perte directement depuis le score.

Softmax

Fonction qui transforme K scores réels en K nombres positifs de somme 1 : $\text{softmax}(z)_k = e^{z_k} / \sum_{r=1}^K e^{z_r}$. Elle généralise la sigmoïde au cas de K classes. Son dénominateur, commun aux K composantes, entraîne

$$\frac{e^{z_a}}{\sum_r e^{z_r}} > \frac{e^{z_b}}{\sum_r e^{z_r}} \iff e^{z_a} > e^{z_b} \iff z_a > z_b,$$

de sorte que softmax et les scores bruts désignent le même argmax.

Loi catégorielle

Loi d'une variable à K issues, de paramètres $\pi_1, \dots, \pi_K$ positifs et de somme 1. Elle généralise Bernoulli au-delà de deux issues. Une régression logistique multinomiale modélise directement une loi catégorielle par softmax ; les K sorties OvR du chapitre 6 prennent une somme libre.

Vraisemblance

Probabilité des données observées, lue comme fonction des paramètres plutôt que des données. Les observations sont fixées, θ varie. Estimer par *maximum de vraisemblance* consiste à retenir la valeur de θ qui rend les étiquettes observées les plus probables. Le logarithme transforme le produit sur les observations en somme, sans déplacer le maximiseur puisque le logarithme croît strictement.

Log-perte, ou entropie croisée binaire

Opposé de la log-vraisemblance moyenne : $-\frac{1}{m} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$. Chaque observation contribue par $-\log$ de la probabilité que le modèle attribuait à l'issue effectivement survenue. Une prédiction correcte et assurée coûte presque 0 ; une prédiction erronée et assurée coûte arbitrairement cher. Maximiser la vraisemblance et minimiser la log-perte sont le même problème.

9.8 Objets de l'optimisation

Gradient

Vecteur des dérivées partielles d'une fonction à plusieurs variables : $\nabla J = (\partial J / \partial \theta_0, \dots, \partial J / \partial \theta_n)$. Chaque composante indique de combien J varie quand un seul coefficient bouge. Le vecteur complet pointe dans la direction de plus forte croissance locale, et son opposé donne la direction de descente employée au chapitre 5.

Descente de gradient

Méthode itérative qui répète $\theta \leftarrow \theta - \alpha \nabla J(\theta)$. Le *taux d'apprentissage* α fixe la longueur du pas. La version dite *par lot* calcule le gradient sur toutes les observations à chaque itération.

Hessienne

Matrice des dérivées secondes, notée $\nabla^2 J$, de taille $(n+1) \times (n+1)$. Son terme (j, l) vaut $\partial^2 J / \partial \theta_j \partial \theta_l$. Là où le gradient donne la pente, la Hessienne donne la courbure : elle dit comment la pente elle-même varie.

L'expression $\mathbf{X}^\top W \mathbf{X}$

Forme prise par la Hessienne de la log-perte logistique, avec $W = \text{diag}(p_i(1-p_i))$ diagonale de taille $m \times m$. Lecture des dimensions : $\mathbf{X}^\top$ est $(n+1) \times m$, W est $m \times m$, $\mathbf{X}$ est $m \times (n+1)$, le produit est donc $(n+1) \times (n+1)$. Chaque observation i pèse $p_i(1-p_i)$, quantité maximale en $p_i = 1/2$ et proche de 0 pour une prédiction assurée : les observations dont le modèle hésite portent la courbure. Cette matrice est l'*information observée* au sens de Fisher, à un facteur $1/m$ près.

Matrice positive semi-définie

Matrice symétrique A telle que $v^\top A v \geq 0$ pour tout vecteur v . C'est l'analogie matriciel de « nombre positif ou nul ». Une fonction dont la Hessienne est positive semi-définie en tout point est convexe. Avec $A = \mathbf{X}^\top W \mathbf{X}$, le calcul donne $v^\top A v = \sum_i p_i(1-p_i)(\tilde{\mathbf{x}}_i^\top v)^2$, somme de termes positifs.

Convexité et stricte convexité

Une fonction est *convexe* lorsque le segment reliant deux points de son graphe passe au-dessus du graphe. Conséquence utile : tout minimum local est global, et une descente locale mène au minimum global. Elle est *strictement convexe* lorsque ce segment passe strictement au-dessus, ce qui rend le minimiseur unique. Une fonction convexe sans stricte convexité possède un fond plat contenant plusieurs minimiseurs équivalents.

Point stationnaire

Point où le gradient s'annule. Pour une fonction convexe, un point stationnaire fini est un minimum global. Pour une fonction quelconque, il peut être un minimum, un maximum ou un point d'inflexion.

Norme euclidienne

$\|v\|_2 = \sqrt{v_1^2 + \dots + v_d^2}$, la longueur du vecteur v au sens ordinaire. Son carré $\|v\|_2^2$ sert de pénalisation au chapitre 4 et sa valeur mesure l'amplitude d'un gradient dans le critère d'arrêt.

Régularisation L^2

Ajout de $\frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$ au coût. Le terme croît avec l'amplitude des coefficients, ce qui déplace l'optimum vers des solutions plus petites. La somme démarre à $j = 1$ et laisse le biais θ_0 libre. On la rencontre aussi sous les noms de *ridge* ou de *weight decay*.

Séparation complète

Situation où un hyperplan classe correctement toutes les observations d'apprentissage. Multiplier les coefficients par un facteur croissant rend alors les probabilités toujours plus extrêmes et la vraisemblance toujours plus grande, ce qui envoie l'optimum à l'infini. La régularisation L^2 ramène l'optimum à distance finie.

Différence finie centrée

Approximation numérique d'une dérivée par $(J(\theta + he_j) - J(\theta - he_j))/(2h)$, avec h petit et e_j le vecteur unité de la coordonnée j . Comparer cette valeur au gradient analytique teste le code du gradient.

Conditionnement

Sensibilité d'un calcul aux directions de l'espace des paramètres. Des colonnes d'échelles très différentes allongent les courbes de niveau du coût, ce qui force un pas court dans une direction pour rester stable dans l'autre. La standardisation ramène les échelles à 1 et arrondit ces courbes.

9.9 Statistiques descriptives

Médiane

Valeur qui partage un échantillon trié en deux moitiés de même effectif. Pour n impair, c'est l'élément central; pour n pair, la moyenne des deux éléments centraux. Elle résiste aux valeurs extrêmes, là où la moyenne se déplace avec elles : sur $\{4, 7, 9, 12, 100\}$, la médiane vaut 9 et la moyenne 26,4.

Écart-type

$s = \sqrt{\frac{1}{m} \sum_i (r_i - \mu)^2}$, dispersion moyenne autour de la moyenne μ , exprimée dans l'unité des données. Le diviseur $m - 1$ donne l'estimateur sans biais de la variance; l'écart entre les deux conventions décroît comme $1/m$.

Standardisation

Transformation $x = (r - \mu)/s$, qui donne à chaque colonne une moyenne de 0 et un écart-type de 1. La valeur obtenue se lit en nombre d'écarts-types au-dessus ou au-dessous de la moyenne. Elle conserve l'ordre des observations et les rapports d'écarts. On la trouve aussi sous le nom de *score z*.

Corrélation linéaire

Coefficient de Pearson $r \in [-1, 1]$, mesurant l'alignement de deux colonnes sur une droite. Une valeur de $+1$ ou -1 signale une relation affine exacte; une valeur proche de 0 signale l'absence de tendance linéaire. Il se calcule comme la covariance divisée par le produit des écarts-types.

Colinéarité

Relation affine exacte ou approchée entre colonnes de la matrice de design. Deux colonnes proportionnelles rendent $X^T W X$ singulière : leurs deux coefficients admettent une infinité de valeurs donnant les mêmes scores. Les prédictions restent uniques, les coefficients cessent d'être identifiables séparément.

Échantillonnage stratifié

Découpage qui applique la proportion voulue à l'intérieur de chaque classe, puis réunit les morceaux. Les effectifs relatifs des classes se retrouvent donc à l'identique dans l'apprentissage et dans le test.

Graine

Entier qui initialise un générateur pseudo-aléatoire. Reprendre le même entier reproduit exactement la même suite de tirages, donc le même mélange et le même découpage.

Hyperparamètre

Quantité fixée par le protocole avant l'ajustement, par opposition aux paramètres que le gradient estime. α , λ , ε et le nombre maximal d'itérations sont des hyperparamètres ; les coefficients θ sont des paramètres.

9.10 Tests et mesures d'évaluation

Risque de généralisation

$R(f) = \mathbb{P}(f(X) \neq C)$, probabilité qu'une règle f se trompe sur une observation tirée de la même loi que les données. La loi étant inconnue, on estime $R(f)$ par la fréquence d'erreur sur un échantillon de test tenu à l'écart de l'ajustement.

Matrice de confusion

Tableau $K \times K$ dont la case (a, b) compte les observations de classe réelle a prédites en b . La diagonale porte les décisions correctes. Les sommes de lignes donnent les effectifs réels, les sommes de colonnes les effectifs prédits.

Exactitude

Proportion de décisions correctes, soit la somme diagonale de la matrice de confusion divisée par l'effectif total. Le terme anglais est *accuracy*.

Rappel et précision

Pour une classe k , le *rappel* vaut $TP/(TP + FN)$, la part des membres réels de k que le modèle retrouve : c'est la diagonale divisée par la somme de la ligne. La *précision* vaut $TP/(TP + FP)$, la part des prédictions k qui sont justes : la diagonale divisée par la somme de la colonne. Deux dénominateurs, deux questions.

Score F_1 et moyenne harmonique

$F_1 = 2PR/(P+R)$, moyenne harmonique de la précision et du rappel. La moyenne harmonique reste proche de la plus petite des deux valeurs, ce qui demande que les deux soient élevées : $P = 0,80$ et $R = 0,50$ donnent $F_1 \approx 0,615$, quand la moyenne arithmétique donnerait 0,65.

Moyenne macro et moyenne pondérée

La moyenne *macro* attribue le même poids à chaque classe. La moyenne *pondérée* pèse chaque classe par son effectif. L'écart entre les deux mesure la part de la classe majoritaire dans le résultat global.

Validation croisée et pli

Un *pli* est un bloc d'observations. La validation croisée à q plis découpe les données en q blocs disjoints, puis ajuste q modèles, chacun laissant un bloc de côté pour la mesure. Chaque observation est ainsi évaluée une fois par un modèle qui ne l'a pas vue.

Test du χ^2 d'indépendance

Test appliqué à un tableau de contingence croisant deux variables qualitatives. Sous l'hypothèse d'indépendance, l'effectif attendu d'une case vaut $E = (\text{total ligne} \times \text{total colonne})/N$. La statistique $\chi^2 = \sum (O - E)^2/E$ mesure l'écart global entre observé et attendu ; elle croît avec le désaccord. Le nombre de *degrés de liberté* vaut $(r - 1)(c - 1)$ pour un tableau à r lignes et

c colonnes, et fixe le seuil de comparaison : 7,81 à 3 degrés de liberté et au niveau 5 %, 25,0 à 15 degrés de liberté. Une statistique inférieure au seuil laisse les données compatibles avec l'indépendance.

Erreur type

Écart-type de la loi d'échantillonnage d'une statistique : de combien elle varierait d'un échantillon à l'autre. Pour une différence de deux moyennes exprimée en écarts-types, elle vaut $\sqrt{1/n_1 + 1/n_2}$. Un écart observé inférieur à deux erreurs types reste compatible avec un effet nul.

Contrôle par permutation

Réajustement du modèle après mélange aléatoire des étiquettes entre les lignes, les caractéristiques restant en place. Le lien entre variables et réponse étant détruit, la performance retombe au niveau de la meilleure règle constante. Un résultat supérieur signale une fuite de données ou une erreur de protocole.

Calibration

Propriété d'un modèle dont les probabilités annoncées correspondent aux fréquences observées : parmi les cas annoncés à 0,8, environ 80 % appartiennent effectivement à la classe. Elle est plus exigeante que le simple classement correct.

Fuite de données

Passage d'information de l'ensemble de test vers l'apprentissage, par exemple en estimant moyennes, écarts-types ou médianes sur le fichier complet. Elle rend l'évaluation optimiste.

9.11 Correspondance avec les notations du sujet

L'annexe mathématique du sujet donne les quatre formules du modèle. Elles sont reproduites ici telles qu'elles y figurent :

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^i \log(h_\theta(x^i)) + (1 - y^i) \log(1 - h_\theta(x^i)) \right],$$

$$h_\theta(x) = g(\theta^T x), \quad g(z) = \frac{1}{1 + e^{-z}}, \quad \frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^i) - y^i) x_j^i.$$

Ce document emploie des noms différents pour les mêmes objets. La table 9.7 donne la correspondance terme à terme.

Les deux écritures du gradient méritent une lecture appariée. Le sujet donne une composante à la fois :

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (p_i - y_i) x_{ij},$$

somme sur les m observations du résidu $p_i - y_i$ multiplié par la caractéristique j . L'équation (2) réunit les $n + 1$ composantes en une seule opération :

$$\nabla J(\theta) = \frac{1}{m} \mathbf{X}^\top (\sigma(\mathbf{X}\theta) - \mathbf{y}).$$

sujet	ce document	objet désigné
$g(z)$	$\sigma(z)$	fonction logistique
$h_{\theta}(x)$	$p_{\theta}(\mathbf{x})$	probabilité prédite pour la classe positive
$\theta^T x$	$\boldsymbol{\theta}^T \tilde{\mathbf{x}}$	score réel, produit scalaire des paramètres et de la ligne augmentée
x^i	$\mathbf{x}_i$	vecteur des caractéristiques de l'observation i
x_j^i	x_{ij}	caractéristique j de l'observation i
y^i	y_i	cible binaire de l'observation i
$J(\theta)$	$J(\boldsymbol{\theta})$	log-perte moyenne, équation (1)
$\partial J / \partial \theta_j$	$(\nabla J(\boldsymbol{\theta}))_j$	composante j du gradient, équation (2)

TABLE 9.7 – Le sujet place l'indice d'observation en exposant, ce document en indice. Les objets et les formules sont identiques.

Le produit $\mathbf{X}^T(\cdot)$ effectue exactement les $n + 1$ sommes précédentes, puisque la ligne j de $\mathbf{X}^T$ contient les x_{ij} de toutes les observations. Un programme peut donc écrire l'une ou l'autre forme ; la seconde laisse la bibliothèque de calcul matriciel exécuter les boucles.

Chapitre 10

Guide de lecture et ressources ouvertes

Les ressources ci-dessous sont ordonnées par objectif et par niveau mathématique. Chaque intitulé en couleur est un lien vers une version ouverte ou vers la page officielle de la ressource.

10.1 Premiers repères statistiques

[Introduction to Modern Statistics – chapitre 9](#)

Ressource libre de Çetinkaya-Rundel et Hardin [8]. Le chapitre sur la régression logistique part de données, explique les cotes et l'interprétation des coefficients avant de développer l'inférence. C'est le point d'entrée de cette liste, à lire en premier pour aborder les probabilités conditionnelles et les log-cotes.

[An Introduction to Statistical Learning](#)

Ouvrage de James, Witten, Hastie, Tibshirani et Taylor [9], disponible gratuitement en versions R et Python. Le traitement est moins technique que *The Elements of Statistical Learning*, avec des exemples, des figures et des laboratoires. Lire le chapitre Classification, puis les chapitres Resampling Methods et Linear Model Selection and Regularization pour comprendre validation croisée et pénalisation.

10.2 Probabilités et calcul

[Stanford CS109 – Logistic Regression](#)

Cours court accompagné de diapositives et de code [10]. Il reprend maximum de vraisemblance et régression logistique dans une progression de probabilités appliquées, au niveau d'un cours d'introduction.

[Stanford Engineering Everywhere – CS229](#)

Le cours historique fournit vidéos, transcriptions, exercices et notes. Les notes 1 couvrent classification et régression logistique ; les notes 5 couvrent régularisation et sélection de modèle [11]. La densité mathématique est supérieure aux deux ressources précédentes, mais les dérivations sont directement utilisables.

Penn State STAT 504 — Analysis of Discrete Data

Notes de cours ouvertes du département de statistique de Penn State [12]. Les leçons 6 à 8 traitent successivement régression logistique binaire, diagnostics et régression multinomiale. Cette ressource est particulièrement utile pour approfondir rapports de cotes, qualité d’ajustement et interprétation statistique.

10.3 Approfondir le formalisme

The Elements of Statistical Learning

Référence de Hastie, Tibshirani et Friedman [1]. Le chapitre 4 contient les équations citées dans ce document. Il est concis et exigeant : sa lecture suit utilement une première approche par ISL ou OpenIntro, puis permet de vérifier une à une vraisemblance, gradient, Hessienne et régularisation.

Probabilistic Machine Learning: An Introduction

Ouvrage de Murphy [7]. Il replace la régression logistique dans un cadre probabiliste plus large et développe optimisation, calibration et modèles multiclassés. À consulter lorsque les objets du présent cours sont déjà maîtrisés séparément.

Pattern Recognition and Machine Learning

Ouvrage de Bishop [13]. Sa section 4.3 traite les modèles linéaires de classification depuis un point de vue probabiliste, et développe la méthode de Newton–Raphson comme alternative à la descente de gradient. C’est la référence à consulter pour dépasser la descente du premier ordre employée ici.

Convex Optimization

Ouvrage ouvert de Boyd et Vandenberghe [4]. Les chapitres sur la convexité et les méthodes de descente donnent le cadre général derrière la Hessienne positive et le choix d’une direction de descente. Cette référence traite l’optimisation mathématique générale ; son étude suppose donc déjà le contexte statistique de la classification.

Parcours conseillé

Pour une première étude : OpenIntro, puis ISL, puis les notes CS229. Pour vérifier le formalisme du présent document : ESL chapitre 4 et Boyd chapitre 9. Pour aller au-delà d’OvR vers une modélisation probabiliste multiclassés : Penn State leçon 8, puis Murphy. Chaque source occupe ainsi un seul rôle : introduction, manuel de calcul ou référence théorique.

Conclusion

Toute la construction tient en une chaîne : un score linéaire devient une probabilité par la fonction logistique, cette probabilité donne une vraisemblance, et maximiser cette vraisemblance revient à faire descendre une fonction convexe dont la pente se calcule exactement. Aucun maillon ne dépend du nombre de classes, ce qui autorise OvR à réutiliser le même problème binaire K fois et à décider par le score maximal. Cette simplicité a une contrepartie précise : la somme des sorties indépendantes prend une valeur libre, quand une distribution catégorielle vaut 1. La validité du résultat tient enfin autant au protocole expérimental — séparation stricte du test, diagnostics par classe, archivage — qu'à la correction des équations : les deux se contrôlent séparément et échouent séparément.

Enchaînement des opérations

Le modèle reçoit des caractéristiques préparées, calcule quatre combinaisons linéaires et retient le score maximal. Son apprentissage choisit chaque vecteur de paramètres de façon à maximiser la vraisemblance de la cible binaire associée. Sa validation porte sur trois questions séparées : la convergence numérique de l'optimisation, la généralisation hors échantillon, et les erreurs propres à chaque maison. Chacune se mesure par ses propres quantités.

Annexe A

Dérivations mathématiques

Cette annexe rassemble les justifications algébriques des formules employées dans le corps du document. Les chapitres principaux en gardent les expressions et leur interprétation opérationnelle.

A.1 Inversion du logit

En posant $p = \mathbb{P}(Y = 1 \mid X = \mathbf{x})$ et $z = \boldsymbol{\theta}^\top \tilde{\mathbf{x}}$,

$$\begin{aligned} \log \frac{p}{1-p} = z &\iff \frac{p}{1-p} = e^z \iff p = e^z(1-p) \\ &\iff p(1 + e^z) = e^z \iff p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}. \end{aligned}$$

A.2 Gradient de la log-perte

Pour une observation, $\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$ et $\partial p_i / \partial z_i = p_i(1 - p_i)$. La règle de chaîne donne

$$\begin{aligned} \frac{\partial \ell_i}{\partial z_i} &= -\frac{y_i}{p_i} p_i(1 - p_i) + \frac{1 - y_i}{1 - p_i} p_i(1 - p_i) \\ &= p_i - y_i. \end{aligned}$$

Comme $\nabla_{\boldsymbol{\theta}} z_i = \tilde{\mathbf{x}}_i$, la moyenne des contributions vaut

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (p_i - y_i) \tilde{\mathbf{x}}_i = \frac{1}{m} \mathbf{X}^\top (\sigma(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}).$$

A.3 Convexité de la log-perte

Avec $W = \text{diag}(p_i(1 - p_i))$,

$$\nabla^2 J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^\top W \mathbf{X}.$$

Pour tout $v \in \mathbb{R}^{n+1}$,

$$v^\top \nabla^2 J(\boldsymbol{\theta}) v = \frac{1}{m} (\mathbf{X}v)^\top W (\mathbf{X}v) = \frac{1}{m} \sum_{i=1}^m p_i(1 - p_i) (\tilde{\mathbf{x}}_i^\top v)^2 \geq 0.$$

La Hessienne est donc positive semi-définie et J est convexe.

A.4 Direction de descente

Pour un déplacement suffisamment petit $\Delta\boldsymbol{\theta}$,

$$J(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \nabla J(\boldsymbol{\theta})^\top \Delta\boldsymbol{\theta} + o(\|\Delta\boldsymbol{\theta}\|).$$

Avec $\Delta\boldsymbol{\theta} = -\alpha \nabla J(\boldsymbol{\theta})$,

$$J(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) = J(\boldsymbol{\theta}) - \alpha \|\nabla J(\boldsymbol{\theta})\|_2^2 + o(\alpha \|\nabla J(\boldsymbol{\theta})\|).$$

Le terme principal est négatif pour $\alpha > 0$ lorsque le gradient est non nul.

Références citées

- [1] Trevor HASTIE, Robert TIBSHIRANI et Jerome FRIEDMAN. *The Elements of Statistical Learning : Data Mining, Inference, and Prediction*. 2^e éd. New York : Springer, 2009. DOI : [10 . 1007 / 978 - 0 - 387 - 84858 - 7](https://doi.org/10.1007/978-0-387-84858-7). URL : [https : // hastie . su . domains / ElemStatLearn/](https://hastie.su.domains/ElemStatLearn/) (visité le 15/07/2026).
- [2] David R. COX. « The Regression Analysis of Binary Sequences ». In : *Journal of the Royal Statistical Society : Series B* 20.2 (1958), p. 215-242. DOI : [10 . 1111 / j . 2517 - 6161 . 1958 . tb00292 . x](https://doi.org/10.1111/j.2517-6161.1958.tb00292.x).
- [3] Adelin ALBERT et John A. ANDERSON. « On the Existence of Maximum Likelihood Estimates in Logistic Regression Models ». In : *Biometrika* 71.1 (1984), p. 1-10. DOI : [10 . 1093 / biomet / 71 . 1 . 1](https://doi.org/10.1093/biomet/71.1.1).
- [4] Stephen BOYD et Lieven VANDENBERGHE. *Convex Optimization*. Cambridge University Press, 2004. URL : [https : // web . stanford . edu / ~ boyd / cvxbook/](https://web.stanford.edu/~boyd/cvxbook/) (visité le 15/07/2026).
- [5] Ryan RIFKIN et Aldebaro KLAUTAU. « In Defense of One-Vs-All Classification ». In : *Journal of Machine Learning Research* 5 (2004), p. 101-141. URL : [https : // www . jmlr . org / papers / v5 / rifkin04a . html](https://www.jmlr.org/papers/v5/rifkin04a.html) (visité le 15/07/2026).
- [6] Bianca ZADROZNY et Charles ELKAN. « Transforming Classifier Scores into Accurate Multiclass Probability Estimates ». In : *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2002, p. 694-699. DOI : [10 . 1145 / 775047 . 775151](https://doi.org/10.1145/775047.775151).

Lectures complémentaires

- [7] Kevin P. MURPHY. *Probabilistic Machine Learning : An Introduction*. MIT Press, 2022. URL : <https://probml.github.io/pml-book/book1.html> (visité le 15/07/2026).
- [8] Mine ÇETINKAYA-RUNDEL et Johanna HARDIN. *Introduction to Modern Statistics*. 2024. URL : <https://openintrostat.github.io/ims/> (visité le 15/07/2026).
- [9] Gareth JAMES et al. *An Introduction to Statistical Learning : with Applications in Python*. Springer, 2023. DOI : 10.1007/978-3-031-38747-0. URL : <https://www.statlearning.com/> (visité le 15/07/2026).
- [10] STANFORD UNIVERSITY. *CS109 : Logistic Regression*. Department of Computer Science. 2026. URL : <https://web.stanford.edu/class/archive/cs/cs109/cs109.1264/lectures/20-LogisticRegression/> (visité le 15/07/2026).
- [11] Andrew NG. *CS229 Lecture Notes : Supervised Learning*. Stanford University, 2018. URL : https://cs229.stanford.edu/main_notes.pdf (visité le 15/07/2026).
- [12] PENNSYLVANIA STATE UNIVERSITY. *STAT 504 : Analysis of Discrete Data*. Department of Statistics. URL : <https://online.stat.psu.edu/stat504/> (visité le 15/07/2026).
- [13] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. New York : Springer, 2006. ISBN : 978-0-387-31073-2.